importing pandas

In [1]:
```
import pandas as pd
import numpy as np
```

Create Series from a List

In [1]:
```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
print(type(s))
```

```
0    10
1    20
2    30
3    40
dtype: int64
<class 'pandas.core.series.Series'>
```

Series from a List with Custom Index

In [3]:
```
s = pd.Series([85, 90, 78], index=["Maths", "Science", "English"])
print(s)
```

```
Maths      85
Science    90
English    78
dtype: int64
```

Create Series from a Tuple

In [ ]:
```
s = pd.Series((5, 10, 15))
print(s)
print(type(s))
```

Create Series from a Dictionary

Keys → Index, Values → Data

In [4]:
```
data = {"A": 100, "B": 200, "C": 300}
s = pd.Series(data)
print(s)
```

```
A    100
B    200
C    300
dtype: int64
```

Series from Dictionary with Specified Index

In [5]:
```
data = {"A": 10, "B": 20, "C": 30}
s = pd.Series(data, index=["A", "C", "D"])
print(s)
```

Vishal Acharya

```
A    10.0
C    30.0
D     NaN
dtype: float64
```

Create Series Using Scalar Value

(Same value repeated)

In [6]:
```
s = pd.Series(5, index=["a", "b", "c", "d"])
print(s)
```

```
a    5
b    5
c    5
d    5
dtype: int64
```

Create Series Using NumPy Array¶

In [7]:
```
import numpy as np
import pandas as pd

arr = np.array([11, 22, 33])
s = pd.Series(arr)
print(s)
```

```
0    11
1    22
2    33
dtype: int32
```

Series with Data Type (dtype)

In [8]:
```
s = pd.Series([1, 2, 3], dtype="float")
print(s)
```

```
0    1.0
1    2.0
2    3.0
dtype: float64
```

Create Empty Series

(Not recommended normally)

In [9]:
```
s = pd.Series(dtype=int)
print(s)
```

```
Series([], dtype: int32)
```

Series from Range

In [10]:
```
s = pd.Series(range(1, 6))
print(s)
```

Vishal Acharya

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

Series from String Characters¶

In [11]:
```python
s = pd.Series(list("DATA"))
print(s)
```

```
0    D
1    A
2    T
3    A
dtype: object
```

Series Using Index Only (NaN values)

In [12]:
```python
s = pd.Series(index=["x", "y", "z"])
print(s)
```

```
x   NaN
y   NaN
z   NaN
dtype: float64
```

6Pandas Series with Name¶

In [13]:
```python
import pandas as pd

s = pd.Series([10, 20, 30], name="Scores")
print(s)
print(s.name)
```

```
0    10
1    20
2    30
Name: Scores, dtype: int64
Scores
```

How to Create a DataFrame in Pandas

Method	Source
Dictionary of lists	Most common
List of lists	Simple data
List of dicts	Uneven data
Dict of Series	Indexed data
NumPy array	Numerical data
CSV file	Real datasets
Empty DataFrame	Initialization

Vishal Acharya

1. Create DataFrame from a Dictionary of Lists (Most Common)

Keys → column names

Values → column data

In [1]:
```
import pandas as pd

data = {
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92],
    "Grade": ["B", "A", "A+"]
}

df = pd.DataFrame(data)
print(df)
```

```
   Name  Marks Grade
0  Amit     78     B
1 Rahul     85     A
2  Neha     92    A+
```

2. Create DataFrame from a List of Lists

In [2]:
```
data = [
    ["Amit", 78],
    ["Rahul", 85],
    ["Neha", 92]
]

df = pd.DataFrame(data, columns=["Name", "Marks"])
print(df)
```

```
   Name  Marks
0  Amit     78
1 Rahul     85
2  Neha     92
```

3. Create DataFrame from a List of Dictionaries

Missing values automatically become NaN

In [3]:
```
data = [
    {"Name": "Amit", "Marks": 78},
    {"Name": "Rahul", "Marks": 85},
    {"Name": "Neha", "Marks": 92}
]

df = pd.DataFrame(data)
print(df)
```

```
   Name  Marks
0  Amit     78
1 Rahul     85
2  Neha     92
```

4. Create DataFrame from a Dictionary of Series

Vishal Acharya

In [4]:
```
s1 = pd.Series([78, 85, 92], index=["A", "B", "C"])
s2 = pd.Series(["B", "A", "A+"], index=["A", "B", "C"])

df = pd.DataFrame({"Marks": s1, "Grade": s2})
print(df)
```

```
   Marks Grade
A     78     B
B     85     A
C     92    A+
```

5. Create DataFrame from a NumPy Array

In [6]:
```
import numpy as np
import pandas as pd

arr = np.array([[1, 2], [3, 4], [5, 6]])
df = pd.DataFrame(arr, columns=["Col1", "Col2"])
print(df)
```

```
   Col1  Col2
0     1     2
1     3     4
2     5     6
```

6. Create DataFrame from a CSV File

In [1]:
```
%%writefile data.csv
maths,sci,phy
25, 21, 18
27, 24, 20
```

```
Writing data.csv
```

In [10]:
```
df = pd.read_csv("data.csv")
print(df)
```

```
   maths  sci  phy
0     25   21   18
1     27   24   20
```

7. Create DataFrame with Custom Index

In [11]:
```
df = pd.DataFrame(
    [[10, 20], [30, 40]],
    columns=["A", "B"],
    index=["Row1", "Row2"]
)
print(df)
```

```
      A   B
Row1  10  20
Row2  30  40
```

8. Create an Empty DataFrame

In [12]:
```
df = pd.DataFrame()
print(df)
```

Vishal Acharya

```
Empty DataFrame
Columns: []
Index: []
```

set_index()?

set_index() is used to convert one or more existing columns into the DataFrame index (row labels).

After setting an index, that column becomes the identifier for rows.

Why use set_index()?

1. Meaningful row labels (IDs, names, dates)
2. Faster lookups with .loc
3. Better data organization
4. Required for time-series & joins

- Basic Syntax: df.set_index(keys, drop=True, inplace=False)

Parameter	Meaning
`keys`	Column name(s) to set as index
`drop`	Remove column after indexing (default `True`)
`inplace`	Modify original DataFrame
`verify_integrity`	Check duplicate index values
`append`	Add to existing index (MultiIndex)

set a Single Column as Index (Most Common)

ID becomes the index

ID column is removed (default behavior)

In [18]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92]
})

print(df)
print("id become index")
df2 = df.set_index("ID")
print(df2)
```

Vishal Acharya

```
    ID   Name  Marks
0  101   Amit     78
1  102  Rahul     85
2  103   Neha     92
id become index
      Name  Marks
ID
101   Amit     78
102  Rahul     85
103   Neha     92
```

Set Index without Dropping the Column

ID stays both as column and index

In [19]:
```python
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92]
})

print(df)
print("ID stays both as column and index")
df2 = df.set_index("ID", drop=False)
print(df2)
```

```
    ID   Name  Marks
0  101   Amit     78
1  102  Rahul     85
2  103   Neha     92
ID stays both as column and index
      ID   Name  Marks
ID
101  101   Amit     78
102  102  Rahul     85
103  103   Neha     92
```

Modify Original DataFrame (inplace=True)

Original DataFrame is changed

In [20]:
```python
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92]
})

print(df)
print("original dataframe changed")
df.set_index("Name", inplace=True)
print(df)
```

Vishal Acharya

```
    ID   Name  Marks
0  101   Amit     78
1  102  Rahul     85
2  103   Neha     92
original dataframe changed
        ID  Marks
Name
Amit   101     78
Rahul  102     85
Neha   103     92
```

set Multiple Columns as Index (MultiIndex)

Creates a hierarchical (multi-level) index

In [21]:
```
df = pd.DataFrame({
    "Year": [2022, 2022, 2023],
    "Team": ["RCB", "MI", "RCB"],
    "Runs": [500, 450, 600]
})

df2 = df.set_index(["Year", "Team"])
print(df2)
```

```
           Runs
Year Team
2022 RCB    500
     MI     450
2023 RCB    600
```

Check for Duplicate Index Values

Raises error if duplicates exist

In [28]:
```
import pandas as pd

df = pd.DataFrame({
    "id": [101, 102, 103],
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92]
})

print(df)
df2 = df.set_index("Name", verify_integrity=True)
print(df2)
```

```
    id   Name  Marks
0  101   Amit     78
1  102  Rahul     85
2  103   Neha     92
        id  Marks
Name
Amit   101     78
Rahul  102     85
Neha   103     92
```

Append to Existing Index

Adds a new index level instead of replacing

Vishal Acharya

In [29]:
```python
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92]
})

print(df)
df = df.reset_index()
df2 = df.set_index(["index", "Name"], append=True)
print(df2)
```

```
    ID   Name  Marks
0  101   Amit     78
1  102  Rahul     85
2  103   Neha     92
                ID  Marks
  index Name
0 0     Amit   101     78
1 1     Rahul  102     85
2 2     Neha   103     92
```

Use set_index() with .loc (Very Important)

Fast and readable row access

In [19]:
```python
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Amit", "Rahul", "Neha"],
    "Marks": [78, 85, 92]
})

print(df)
df = df.reset_index(drop=False)
print(df)
df = df.set_index("ID")
print(df)

print(df.loc[102])
```

Vishal Acharya

```
    ID   Name  Marks
0  101   Amit     78
1  102  Rahul     85
2  103   Neha     92
   index   ID   Name  Marks
0      0  101   Amit     78
1      1  102  Rahul     85
2      2  103   Neha     92
     index   Name  Marks
ID
101      0   Amit     78
102      1  Rahul     85
103      2   Neha     92
index        1
Name     Rahul
Marks       85
Name: 102, dtype: object
```

set_index() vs reset_index()

Method	Use
`set_index()`	Column → Index
`reset_index()`	Index → Column

set_index() → Sets one or more columns as the DataFrame index (column becomes row label)

drop=True → Removes column after indexing

inplace=True → Modifies original DataFrame

DataFrame attributes?

DataFrame attributes are properties that give information about the structure, size, labels, and data types of a DataFrame.

Attributes do NOT use parentheses ().

Attribute	Purpose
`shape`	Rows & columns
`size`	Total elements
`ndim`	Dimensions
`columns`	Column names
`index`	Row labels
`dtypes`	Data types
`values`	NumPy array
`T`	Transpose

Vishal Acharya

Attribute	Purpose
`empty`	Empty check
`axes`	All axes
`memory_usage()(method)`	Memory info

df.shape

Definition: Returns a tuple (rows, columns) of the DataFrame.

Purpose

Know dataset size

Used before analysis

Code: print(df.shape)

Meaning → 4 rows, 3 columns

In [45]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})

print(df.shape)
```

```
(4, 3)
```

df.size

Definition: Returns total number of elements (rows × columns).

Purpose

Memory estimation

Data volume check

Code: print(df.size)

In [46]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.size)
```

```
12
```

df.ndim

Vishal Acharya

Definition: Returns number of dimensions of DataFrame.

Purpose: Identify data structure type

Code: print(df.ndim)

DataFrame is always 2D

In [47]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.ndim)
```

```
2
```

df.columns

Definition: Returns column labels of the DataFrame.

Purpose

Column inspection

Column selection

Code: print(df.columns)

In [50]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.columns)
```

```
Index(['ID', 'Name', 'Marks'], dtype='object')
```

df.index

Definition: Returns row labels (index).

Purpose

Row identification

Reindexing

Code:print(df.index)

In [51]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
```

Vishal Acharya

```
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.index)
```

```
RangeIndex(start=0, stop=4, step=1)
```

df.dtypes

Definition: Returns data type of each column.

Purpose

Data validation

Type conversion planning

Code: print(df.dtypes)

In [52]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.dtypes)
```

```
ID         int64
Name      object
Marks    float64
dtype: object
```

df.values

Definition: Returns data as a NumPy array (without labels).

Purpose

NumPy operations

Fast computation

Code: print(df.values)

In [53]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})

print(df.values)
```

```
[[101 'Amit' 78.0]
 [102 'Rahul' 85.0]
 [103 'Neha' 92.0]
 [104 'Priya' nan]]
```

Vishal Acharya

df.T (Transpose)

Definition: Swaps rows and columns.

Purpose

Data orientation change

Mathematical operations

Code: print(df.T)

In [54]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})

print(df)
print("swap")
print(df.T)
```

```
    ID   Name  Marks
0  101   Amit   78.0
1  102  Rahul   85.0
2  103   Neha   92.0
3  104  Priya    NaN
swap
          0      1     2      3
ID      101    102   103    104
Name   Amit  Rahul  Neha  Priya
Marks  78.0   85.0  92.0    NaN
```

df.empty

Definition: Checks whether DataFrame is empty.

Purpose: Validation before processing

Code:print(df.empty)

In [55]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.empty)
```

```
False
```

df.axes

Definition: Returns both row index and column index.

Purpose: Full axis inspection

Vishal Acharya

Code: print(df.axes)

In [56]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.axes)
```

```
[RangeIndex(start=0, stop=4, step=1), Index(['ID', 'Name', 'Marks'], dtype='objec
t')]
```

df.memory_usage()

Definition: Shows memory usage of each column.

Purpose

Performance optimization

Large dataset handling

Code: print(df.memory_usage())

In [57]:
```
import pandas as pd

df = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Amit", "Rahul", "Neha", "Priya"],
    "Marks": [78, 85, 92, None]
})
print(df.memory_usage())
```

```
Index    132
ID        32
Name      32
Marks     32
dtype: int64
```

In [2]:
```
%%writefile students.csv
ID,Name,Marks,City
101,Amit,78,Ahmedabad
102,Rahul,85,Surat
103,Neha,92,Baroda
104,Priya,,Rajkot
```

```
Writing students.csv
```

In [59]:
```
import pandas as pd

# Read CSV file
df = pd.read_csv("students.csv")

print("*" * 60)
print("DATAFRAME")
print("*" * 60)
print(df)
```

Vishal Acharya

```python
print("*" * 60)
print("df.shape  → (rows, columns)")
print("*" * 60)
print(df.shape)

print("*" * 60)
print("df.size  → total elements")
print("*" * 60)
print(df.size)

print("*" * 60)
print("df.ndim  → number of dimensions")
print("*" * 60)
print(df.ndim)

print("*" * 60)
print("df.columns  → column names")
print("*" * 60)
print(df.columns)

print("*" * 60)
print("df.index  → row index")
print("*" * 60)
print(df.index)

print("*" * 60)
print("df.dtypes  → data type of each column")
print("*" * 60)
print(df.dtypes)

print("*" * 60)
print("df.values  → NumPy array")
print("*" * 60)
print(df.values)

print("*" * 60)
print("df.T  → transpose (rows ↔ columns)")
print("*" * 60)
print(df.T)

print("*" * 60)
print("df.empty  → check empty DataFrame")
print("*" * 60)
print(df.empty)

print("*" * 60)
print("df.axes  → row & column axes")
print("*" * 60)
print(df.axes)

print("*" * 60)
print("df.memory_usage()  → memory used by columns")
print("*" * 60)
print(df.memory_usage())

print("*" * 60)
print("df.info()  → full DataFrame summary")
print("*" * 60)
df.info()
```

Vishal Acharya

```
************************************************************
DATAFRAME
************************************************************
    ID   Name  Marks       City
0  101   Amit   78.0  Ahmedabad
1  102  Rahul   85.0      Surat
2  103   Neha   92.0     Baroda
3  104  Priya    NaN     Rajkot
************************************************************
df.shape  → (rows, columns)
************************************************************
(4, 4)
************************************************************
df.size  → total elements
************************************************************
16
************************************************************
df.ndim  → number of dimensions
************************************************************
2
************************************************************
df.columns  → column names
************************************************************
Index(['ID', 'Name', 'Marks', 'City'], dtype='object')
************************************************************
df.index  → row index
************************************************************
RangeIndex(start=0, stop=4, step=1)
************************************************************
df.dtypes  → data type of each column
************************************************************
ID         int64
Name      object
Marks    float64
City      object
dtype: object
************************************************************
df.values  → NumPy array
************************************************************
[[101 'Amit' 78.0 'Ahmedabad']
 [102 'Rahul' 85.0 'Surat']
 [103 'Neha' 92.0 'Baroda']
 [104 'Priya' nan 'Rajkot']]
************************************************************
df.T  → transpose (rows ↔ columns)
************************************************************
               0      1       2       3
ID           101    102     103     104
Name        Amit  Rahul    Neha   Priya
Marks       78.0   85.0    92.0     NaN
City   Ahmedabad  Surat  Baroda  Rajkot
************************************************************
df.empty  → check empty DataFrame
************************************************************
False
************************************************************
df.axes  → row & column axes
************************************************************
[RangeIndex(start=0, stop=4, step=1), Index(['ID', 'Name', 'Marks', 'City'], dtyp
e='object')]
```

Vishal Acharya

```
************************************************************
df.memory_usage()  → memory used by columns
************************************************************
Index    132
ID        32
Name      32
Marks     32
City      32
dtype: int64
************************************************************
df.info()  → full DataFrame summary
************************************************************
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      4 non-null      int64
 1   Name    4 non-null      object
 2   Marks   3 non-null      float64
 3   City    4 non-null      object
dtypes: float64(1), int64(1), object(2)
memory usage: 260.0+ bytes
```

What is an Index in a Pandas DataFrame?

Definition: An Index is a label used to identify rows in a DataFrame. By default, Pandas assigns 0, 1, 2, 3, ...

- Purpose

Identify rows uniquely

Fast data access

Enable slicing and alignment

Support meaningful labels (like names, dates) | Operation | Method | | ---------- | ---------- | | Row select (label) | `loc` | | Row select (position) | `iloc` | | Row slicing | `df.loc["A":"C"]` | | Edit value | `df.loc[row, col]` | | Add row | `df.loc[new_index]` | | Add column | `df["new_col"]` |

1. loc → label based
2. iloc → position based
3. loc includes end index
4. iloc excludes end index
5. New row added only with new index
6. Column length must match number of rows

View index

Change index

Vishal Acharya

In [82]:
```python
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
print(df.index)
print("change index")
df.index = ["A", "B", "C", "D"]
print(df)
```

```
RangeIndex(start=0, stop=4, step=1)
change index
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     48.7
C   Gill   3500     51.2
D  Rahul   4200     45.3
```

Indexing in DataFrame (Row & Column Selection)

- Select column (by label)
- Select multiple columns

In [92]:
```python
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
df.index = ["A", "B", "C", "D"]
print(df["runs"])
print(df[["name", "average"]])
```

```
A    12000
B    11000
C     3500
D     4200
Name: runs, dtype: int64
    name  average
A  Kohli     59.5
B  Rohit     48.7
C   Gill     51.2
D  Rahul     45.3
```

Slicing Rows Using Index (Label-based)

df.loc[start_label : end_label]

loc includes the ending index

In [94]:
```python
import pandas as pd
```

Vishal Acharya

```
data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
df.index = ["A", "B", "C", "D"]
print(df)
df.loc["A":"C"]
```

```
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     48.7
C   Gill   3500     51.2
D  Rahul   4200     45.3
```

Out[94]:

	name	runs	average
A	Kohli	12000	59.5
B	Rohit	11000	48.7
C	Gill	3500	51.2

Slicing Rows Using Position (Integer-based)

df.iloc[start_position : end_position]

iloc excludes the ending position

In [96]:

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
print(df)
df.iloc[0:2]
```

```
    name   runs  average
0  Kohli  12000     59.5
1  Rohit  11000     48.7
2   Gill   3500     51.2
3  Rahul   4200     45.3
```

Out[96]:

	name	runs	average
0	Kohli	12000	59.5
1	Rohit	11000	48.7

Editing Data Using Index & Slicing

Edit a single value

Vishal Acharya

In [100…

```python
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]}

df = pd.DataFrame(data,index=["A","B","C","D"])
print(df)
df.loc["C", "runs"] = 4000
print("Edit")
print(df)
```

```
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     48.7
C   Gill   3500     51.2
D  Rahul   4200     45.3
Edit
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     48.7
C   Gill   4000     51.2
D  Rahul   4200     45.3
```

Edit multiple rows (slicing)

In [101…

```python
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data,index=["A","B","C","D"])
print(df)
print("Edit")
df.loc["B":"D", "average"] = 50
print(df)
```

```
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     48.7
C   Gill   3500     51.2
D  Rahul   4200     45.3
Edit
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     50.0
C   Gill   3500     50.0
D  Rahul   4200     50.0
```

Edit multiple columns

In [104…

```python
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
```

Vishal Acharya

```
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data,index=["A","B","C","D"])
print(df)
print("Edit")
df.loc["A":"C", ["runs", "average"]] = [13000, 60]
print(df)
```

```
    name   runs  average
A  Kohli  12000     59.5
B  Rohit  11000     48.7
C   Gill   3500     51.2
D  Rahul   4200     45.3
Edit
    name   runs  average
A  Kohli  13000     60.0
B  Rohit  13000     60.0
C   Gill  13000     60.0
D  Rahul   4200     45.3
```

Editing Using iloc

Change value by position

In [105…

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
df.iloc[1, 1] = 11500
df
```

Out[105…

	name	runs	average
0	Kohli	12000	59.5
1	Rohit	11500	48.7
2	Gill	3500	51.2
3	Rahul	4200	45.3

Change multiple rows

In [107…

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
```

Vishal Acharya

```
df.iloc[0:2, 2] = 55
df
```

Out[107…

	name	runs	average
0	Kohli	12000	55.0
1	Rohit	11000	55.0
2	Gill	3500	51.2
3	Rahul	4200	45.3

Add row using loc

- Key Point

Index label must be new

Order of values must match columns

In [109…

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data,index=["A","B","C","D"])
df.loc["E"] = ["Pant", 3000, 44.5]
df
```

Out[109…

	name	runs	average
A	Kohli	12000	59.5
B	Rohit	11000	48.7
C	Gill	3500	51.2
D	Rahul	4200	45.3
E	Pant	3000	44.5

Add a New Column Using Index & Slicing

Add full column

In [111…

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
```

Vishal Acharya

```
df["matches"] = [280, 260, 70, 90]
print(df)
```

```
   name   runs  average  matches
0  Kohli  12000     59.5      280
1  Rohit  11000     48.7      260
2   Gill   3500     51.2       70
3  Rahul   4200     45.3       90
```

Add column using condition

In [112…

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
df.loc[:, "strike_rate"] = [93.5, 89.2, 92.1, 88.4]
df
```

Out[112…

	name	runs	average	strike_rate
0	Kohli	12000	59.5	93.5
1	Rohit	11000	48.7	89.2
2	Gill	3500	51.2	92.1
3	Rahul	4200	45.3	88.4

add Column Using Existing Data

In [115…

```
import pandas as pd

data = {
    "name": ["Kohli", "Rohit", "Gill", "Rahul"],
    "runs": [12000, 11000, 3500, 4200],
    "average": [59.5, 48.7, 51.2, 45.3]
}

df = pd.DataFrame(data)
df["performance"] = "Good"
print(df)
df.loc[df["average"] < 50, "performance"] = "Average"
df
```

```
   name   runs  average performance
0  Kohli  12000     59.5        Good
1  Rohit  11000     48.7        Good
2   Gill   3500     51.2        Good
3  Rahul   4200     45.3        Good
```

Vishal Acharya

Out[115…

	name	runs	average	performance
0	Kohli	12000	59.5	Good
1	Rohit	11000	48.7	Average
2	Gill	3500	51.2	Good
3	Rahul	4200	45.3	Average

In [116…

```
%%writefile hollywood.csv
movie,genre,hero,collection,rating
Inception,Sci-Fi,Leonardo,829,8.8
Titanic,Romance,Leonardo,2187,7.9
Avengers,Action,Robert,2797,8.4
Joker,Drama,Joaquin,1074,8.5
Avatar,Sci-Fi,Sam,2923,7.8
```

```
Writing hollywood.csv
```

In [117…

```
import pandas as pd

# --------------------------------------
# Read CSV file
# --------------------------------------
df = pd.read_csv("hollywood.csv")
print("Original DataFrame:\n", df)

# --------------------------------------
# View index
# --------------------------------------
print("\nIndex:", df.index)

# --------------------------------------
# Set movie as index
# --------------------------------------
df.set_index("movie", inplace=True)
print("\nAfter setting movie as index:\n", df)

# --------------------------------------
# Column indexing
# --------------------------------------
print("\nSelect single column:\n", df["genre"])

print("\nSelect multiple columns:\n", df[["hero", "rating"]])

# --------------------------------------
# Row indexing using loc (label-based)
# --------------------------------------
print("\nRow using loc:\n", df.loc["Titanic"])

# --------------------------------------
# Row indexing using iloc (position-based)
# --------------------------------------
print("\nRow using iloc:\n", df.iloc[1])

# --------------------------------------
# Row slicing using loc
# --------------------------------------
print("\nRow slicing loc:\n", df.loc["Inception":"Joker"])
```

Vishal Acharya

```python
# --------------------------------------
# Row slicing using iloc
# --------------------------------------
print("\nRow slicing iloc:\n", df.iloc[0:3])

# --------------------------------------
# Editing single value using loc
# --------------------------------------
df.loc["Avatar", "rating"] = 8.2
print('df.loc["Avatar", "rating"] = 8.2')
print(df)
# --------------------------------------
# Editing multiple rows using slicing
# --------------------------------------
df.loc["Inception":"Titanic", "rating"] = 9.0
print('df.loc["Inception":"Titanic", "rating"] = 9.0')
print(df)
# --------------------------------------
# Editing using iloc
# --------------------------------------
df.iloc[2, 3] = 3000   # Change Avengers collection
print("df.iloc[2, 3] = 3000")
print(df)
# --------------------------------------
# Add new column
# --------------------------------------
df["status"] = "Hit"
df.loc[df["rating"] < 8, "status"] = "Average"
print('''df["status"] = "Hit"
df.loc[df["rating"] < 8, "status"] = "Average"''')
print(df)
# --------------------------------------
# Add new row using index
# --------------------------------------
df.loc["Interstellar"] = ["Sci-Fi", "Matthew", 677, 8.6, "Hit"]
print('df.loc["Interstellar"] = ["Sci-Fi", "Matthew", 677, 8.6, "Hit"]')
print(df)
# --------------------------------------
# Final DataFrame
# --------------------------------------
print("\nFinal DataFrame:\n", df)
```

Vishal Acharya

```
Original DataFrame:
        movie    genre      hero  collection  rating
0  Inception   Sci-Fi  Leonardo         829     8.8
1    Titanic  Romance  Leonardo        2187     7.9
2   Avengers   Action    Robert        2797     8.4
3      Joker    Drama   Joaquin        1074     8.5
4     Avatar   Sci-Fi       Sam        2923     7.8

Index: RangeIndex(start=0, stop=5, step=1)

After setting movie as index:
              genre      hero  collection  rating
movie
Inception   Sci-Fi  Leonardo         829     8.8
Titanic    Romance  Leonardo        2187     7.9
Avengers    Action    Robert        2797     8.4
Joker        Drama   Joaquin        1074     8.5
Avatar      Sci-Fi       Sam        2923     7.8

Select single column:
 movie
Inception     Sci-Fi
Titanic      Romance
Avengers      Action
Joker          Drama
Avatar        Sci-Fi
Name: genre, dtype: object

Select multiple columns:
                hero  rating
movie
Inception  Leonardo     8.8
Titanic    Leonardo     7.9
Avengers     Robert     8.4
Joker       Joaquin     8.5
Avatar          Sam     7.8

Row using loc:
 genre          Romance
hero          Leonardo
collection        2187
rating             7.9
Name: Titanic, dtype: object

Row using iloc:
 genre          Romance
hero          Leonardo
collection        2187
rating             7.9
Name: Titanic, dtype: object

Row slicing loc:
              genre      hero  collection  rating
movie
Inception   Sci-Fi  Leonardo         829     8.8
Titanic    Romance  Leonardo        2187     7.9
Avengers    Action    Robert        2797     8.4
Joker        Drama   Joaquin        1074     8.5

Row slicing iloc:
```

Vishal Acharya

```
              genre      hero  collection  rating
movie
Inception   Sci-Fi  Leonardo         829     8.8
Titanic    Romance  Leonardo        2187     7.9
Avengers    Action    Robert        2797     8.4
df.loc["Avatar", "rating"] = 8.2
             genre      hero  collection  rating
movie
Inception   Sci-Fi  Leonardo         829     8.8
Titanic    Romance  Leonardo        2187     7.9
Avengers    Action    Robert        2797     8.4
Joker        Drama   Joaquin        1074     8.5
Avatar      Sci-Fi       Sam        2923     8.2
df.loc["Inception":"Titanic", "rating"] = 9.0
             genre      hero  collection  rating
movie
Inception   Sci-Fi  Leonardo         829     9.0
Titanic    Romance  Leonardo        2187     9.0
Avengers    Action    Robert        2797     8.4
Joker        Drama   Joaquin        1074     8.5
Avatar      Sci-Fi       Sam        2923     8.2
df.iloc[2, 3] = 3000
             genre      hero  collection  rating
movie
Inception   Sci-Fi  Leonardo         829     9.0
Titanic    Romance  Leonardo        2187     9.0
Avengers    Action    Robert        2797  3000.0
Joker        Drama   Joaquin        1074     8.5
Avatar      Sci-Fi       Sam        2923     8.2
df["status"] = "Hit"
df.loc[df["rating"] < 8, "status"] = "Average"
             genre      hero  collection  rating status
movie
Inception   Sci-Fi  Leonardo         829     9.0    Hit
Titanic    Romance  Leonardo        2187     9.0    Hit
Avengers    Action    Robert        2797  3000.0    Hit
Joker        Drama   Joaquin        1074     8.5    Hit
Avatar      Sci-Fi       Sam        2923     8.2    Hit
df.loc["Interstellar"] = ["Sci-Fi", "Matthew", 677, 8.6, "Hit"]
                genre      hero  collection  rating status
movie
Inception      Sci-Fi  Leonardo         829     9.0    Hit
Titanic       Romance  Leonardo        2187     9.0    Hit
Avengers       Action    Robert        2797  3000.0    Hit
Joker           Drama   Joaquin        1074     8.5    Hit
Avatar         Sci-Fi       Sam        2923     8.2    Hit
Interstellar   Sci-Fi   Matthew         677     8.6    Hit

Final DataFrame:
                 genre      hero  collection  rating status
movie
Inception      Sci-Fi  Leonardo         829     9.0    Hit
Titanic       Romance  Leonardo        2187     9.0    Hit
Avengers       Action    Robert        2797  3000.0    Hit
Joker           Drama   Joaquin        1074     8.5    Hit
Avatar         Sci-Fi       Sam        2923     8.2    Hit
Interstellar   Sci-Fi   Matthew         677     8.6    Hit
```

Vishal Acharya

DataFrame methods are functions used to view, clean, analyze, and manipulate tabular data in Pandas.

Attributes = information, Methods = action

Category	Method	Purpose
View	`head()`	Show first rows
	`tail()`	Show last rows
	`sample()`	Show random rows
Inspect	`info()`	Structure, dtype, null count
	`describe()`	Statistical summary
Unique / Frequency	`unique()`	Get unique values
	`nunique()`	Count unique values
	`value_counts()`	Frequency of values
Missing Data	`isnull()`	Detect missing values
	`fillna()`	Replace missing values
	`dropna()`	Remove missing values
Duplicates	`duplicated()`	Mark duplicate rows
	`drop_duplicates()`	Remove duplicate rows
Sorting	`sort_values()`	Sort by column values
	`sort_index()`	Sort by index
Statistics	`sum()`	Total
	`mean()`	Average
	`min()`	Smallest value
	`max()`	Largest value
	`mode()`	Most frequent value
	`std()`	Standard deviation
Transformation	`apply()`	Apply function
	`astype()`	Change data type
	`replace()`	Replace values
	`drop()`	Remove rows/columns
	`rename()`	Rename columns/index
Index Handling	`set_index()`	Column → index
	`reset_index()`	Index → column

Vishal Acharya

Category	Method	Purpose
Utility	`copy()`	Create safe copy
	`corr()`	Correlation between columns
	`count()`	Count non-null values
	`memory_usage()`	Memory consumption info

sample dataframe

In [119…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})

print(df)
```

```
   ID   Name  Marks       City
0   1   Amit   78.0  Ahmedabad
1   2  Rahul   85.0      Surat
2   2  Rahul   85.0      Surat
3   3   Neha    NaN     Baroda
4   4  Priya   90.0     Rajkot
5   4  Priya   90.0     Rajkot
```

head()

Definition: Returns the first n rows of the DataFrame.

Purpose: Quick preview of data.

Syntax: df.head(n=5)

default value of n is 5

In [182…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df.head(2)
```

Out[182…

	ID	Name	Marks	City
0	1	Amit	78.0	Ahmedabad
1	2	Rahul	85.0	Surat

Vishal Acharya

tail()

Definition: Returns the last n rows.

Syntax: df.tail(n=5)

default value of n is 5

In [183…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})

print(df.tail(2))
```

```
   ID   Name  Marks    City
4   4  Priya   90.0  Rajkot
5   4  Priya   90.0  Rajkot
```

sample()

Definition: Returns random rows.

Purpose: Random inspection (avoid bias).

Syntax:df.sample(n=None, frac=None, random_state=None)

Parameter	Meaning
`n`	Number of rows
`frac`	Fraction of rows
`random_state`	Same random output

In [190…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})

print(df.sample())
print(df.sample())
print(df.sample(frac=True))
print(df.sample(frac=False))
```

Vishal Acharya

```
   ID   Name  Marks   City
1   2  Rahul   85.0  Surat
   ID   Name  Marks    City
4   4  Priya   90.0  Rajkot
   ID   Name  Marks       City
4   4  Priya   90.0     Rajkot
0   1   Amit   78.0  Ahmedabad
1   2  Rahul   85.0      Surat
5   4  Priya   90.0     Rajkot
2   2  Rahul   85.0      Surat
3   3   Neha    NaN     Baroda
Empty DataFrame
Columns: [ID, Name, Marks, City]
Index: []
```

INSPECT METHODS

info()

Definition: Displays structure summary.

Purpose: Check:

column names

data types

non-null values

memory usage

Syntax: df.info()

In [191…

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      6 non-null      int64
 1   Name    6 non-null      object
 2   Marks   5 non-null      float64
 3   City    6 non-null      object
dtypes: float64(1), int64(1), object(2)
memory usage: 324.0+ bytes
```

describe()

Vishal Acharya

Definition: Returns statistical summary.

Syntax: df.describe(include=None, exclude=None)

Parameter	Meaning
`include="all"`	Include all columns
`exclude`	Exclude data types

Feature	`include`	`exclude`
Purpose	Select specific data types	Remove specific data types
Accepts	Type or list of types	Type or list of types
Use together?	❌ No	❌ No

Argument	Meaning
`int`, `float`	Only numeric columns
`number`	All numeric types
`object`	String / categorical columns
`bool`	Boolean columns
`datetime`	Date-time columns
`category`	Categorical columns
List	Multiple data types

In [1]:
```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df.describe()
```

Vishal Acharya

Out[1]:

	ID	Marks
count	6.000000	5.000000
mean	2.666667	85.600000
std	1.211060	4.929503
min	1.000000	78.000000
25%	2.000000	85.000000
50%	2.500000	85.000000
75%	3.750000	90.000000
max	4.000000	90.000000

In [2]:
```
df.describe(percentiles=[0.2,0.3])
```

Out[2]:

	ID	Marks
count	6.000000	5.000000
mean	2.666667	85.600000
std	1.211060	4.929503
min	1.000000	78.000000
20%	2.000000	83.600000
30%	2.000000	85.000000
50%	2.500000	85.000000
max	4.000000	90.000000

In [193…
```
df.describe(include="all")
```

Vishal Acharya

Out[193...

	ID	Name	Marks	City
count	6.000000	6	5.000000	6
unique	NaN	4	NaN	4
top	NaN	Rahul	NaN	Surat
freq	NaN	2	NaN	2
mean	2.666667	NaN	85.600000	NaN
std	1.211060	NaN	4.929503	NaN
min	1.000000	NaN	78.000000	NaN
25%	2.000000	NaN	85.000000	NaN
50%	2.500000	NaN	85.000000	NaN
75%	3.750000	NaN	90.000000	NaN
max	4.000000	NaN	90.000000	NaN

In [195...

```
df.describe(exclude="int64")
```

Out[195...

	Name	Marks	City
count	6	5.000000	6
unique	4	NaN	4
top	Rahul	NaN	Surat
freq	2	NaN	2
mean	NaN	85.600000	NaN
std	NaN	4.929503	NaN
min	NaN	78.000000	NaN
25%	NaN	85.000000	NaN
50%	NaN	85.000000	NaN
75%	NaN	90.000000	NaN
max	NaN	90.000000	NaN

In [196...

```
df.describe(exclude=["int64","float64"])
```

Out[196...

	Name	City
count	6	6
unique	4	4
top	Rahul	Surat
freq	2	2

Vishal Acharya

UNIQUE & FREQUENCY METHODS

unique() (Column level)

Definition: Returns unique values. Syntax: df["City"].unique()

In [198…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df["City"].unique()
```

Out[198…

```
array(['Ahmedabad', 'Surat', 'Baroda', 'Rajkot'], dtype=object)
```

nunique()

Definition: returns count of unique values.

Syntax: df["City"].nunique()

In [200…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({

    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
print(df.nunique())
print("series")
print(df["City"].nunique())
```

```
ID       4
Name     4
Marks    3
City     4
dtype: int64
series
4
```

value_counts()

Definition: Returns frequency of values.

Syntax: df["City"].value_counts(normalize=False) | Parameter | Meaning | | ---------- | ---------- | | `normalize=True` | Percentage | | `ascending=True` | Sort order |

In [201…

```
import pandas as pd
import numpy as np
```

Vishal Acharya

```
df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df["City"].value_counts(normalize=False)
```

Out[201…

```
City
Surat        2
Rajkot       2
Ahmedabad    1
Baroda       1
Name: count, dtype: int64
```

In [202…

```
df["City"].value_counts(normalize=True)
```

Out[202…

```
City
Surat        0.333333
Rajkot       0.333333
Ahmedabad    0.166667
Baroda       0.166667
Name: proportion, dtype: float64
```

In [204…

```
df.value_counts(ascending=False)
```

Out[204…

```
ID  Name   Marks  City
2   Rahul  85.0   Surat        2
4   Priya  90.0   Rajkot       2
1   Amit   78.0   Ahmedabad    1
Name: count, dtype: int64
```

In [205…

```
df.value_counts(ascending=True)
```

Out[205…

```
ID  Name   Marks  City
1   Amit   78.0   Ahmedabad    1
2   Rahul  85.0   Surat        2
4   Priya  90.0   Rajkot       2
Name: count, dtype: int64
```

MISSING DATA HANDLING

isnull()

definition:Detects missing values.

Syntax:df.isnull()

In [206…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
```

Vishal Acharya

```
print(df.isnull())
print(df.isnull().sum())
```

```
      ID   Name  Marks   City
0  False  False  False  False
1  False  False  False  False
2  False  False  False  False
3  False  False   True  False
4  False  False  False  False
5  False  False  False  False
ID       0
Name     0
Marks    1
City     0
dtype: int64
```

fillna(): fillna() replaces missing values using constants, statistical measures (mean, median, mode), or forward/backward filling methods in pandas.

Definition: Replaces missing values.

Syntax: df.fillna(value=None, method=None, inplace=False)

Parameter	Meaning
`value`	Replace with value
`method="ffill"`	Forward fill
`method="bfill"`	Backward fill
`inplace=True`	Modify original

Fill Missing Values with Mean

Explanation: Mean is best used when data is numerical and normally distributed.

Each column's missing values are replaced with its column mean.

In [210…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Maths": [85, 90, np.nan, 88, np.nan],
    "Science": [78, np.nan, 82, 80, 79]
})

print(df)
df.fillna(df.mean(), inplace=True)
print(df)
```

Vishal Acharya

```
   Maths  Science
0   85.0     78.0
1   90.0      NaN
2    NaN     82.0
3   88.0     80.0
4    NaN     79.0
       Maths  Science
0  85.000000    78.00
1  90.000000    79.75
2  87.666667    82.00
3  88.000000    80.00
4  87.666667    79.00
```

In [211…

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df=df.fillna(df.mean(numeric_only=True))
print(df)
```

```
   ID   Name  Marks       City
0   1   Amit   78.0  Ahmedabad
1   2  Rahul   85.0      Surat
2   2  Rahul   85.0      Surat
3   3   Neha   85.6     Baroda
4   4  Priya   90.0     Rajkot
5   4  Priya   90.0     Rajkot
```

Fill Missing Values with Median

Explanation: Median is preferred when data contains outliers.

Missing values replaced using column median

In [212…

```python
df = pd.DataFrame({
    "Maths": [85, 90, np.nan, 88, np.nan],
    "Science": [78, np.nan, 82, 80, 79]
})

df.fillna(df.median(), inplace=True)
print(df)
```

```
   Maths  Science
0   85.0     78.0
1   90.0     79.5
2   88.0     82.0
3   88.0     80.0
4   88.0     79.0
```

Fill Missing Values with Mode

Explanation: Mode is useful for categorical or repeated numeric values.

✔ df.mode() returns a DataFrame

Vishal Acharya

✔ .iloc[0] selects the first mode

In [214…

```
df = pd.DataFrame({
    "Grade": ["A", "B", None, "A", None]
})

df.fillna(df.mode().iloc[0], inplace=True)
print(df)
```

```
  Grade
0     A
1     B
2     A
3     A
4     A
```

Fill Column-Wise Using Dictionary

In [217…

```
df = pd.DataFrame({
    "Age": [20, None, 22],
    "City": ["Ahmedabad", None, "Surat"]
})

df.fillna({"Age": 21, "City": "Unknown"}, inplace=True)
print(df)
```

```
    Age       City
0  20.0  Ahmedabad
1  21.0    Unknown
2  22.0      Surat
```

Important Notes

1. fillna() works on Series & DataFrame
2. inplace=True permanently changes data
3. Cannot use value and method together
4. Mean/Median → Numerical data
5. Mode → Categorical data

dropna():is used to remove rows or columns that contain missing values (NaN) from a DataFrame or Series.

Definition: Removes rows/columns with NaN.

Syntax: df.dropna(axis=0, how="any", thresh=None, subset=None, inplace=False)

Parameter	Purpose
`axis`	Rows or columns
`how`	Condition to drop
`thresh`	Minimum non-NaN values
`subset`	Specific columns
`inplace`	Modify original

Vishal Acharya

Vishal Acharya

axis

Value	Meaning
`axis=0`	Remove **rows** (default)
`axis=1`	Remove **columns**

how

Value	Meaning
`how="any"`	Drop if **any NaN** present
`how="all"`	Drop only if **all values are NaN**

thresh

Keeps rows/columns having minimum non-NaN values

df.dropna(thresh=2)

- Keeps rows with at least 2 non-NaN values

subset: Consider NaN only in specific columns

inplace:

Value	Meaning
`True`	Modify original DataFrame
`False`	Return new DataFrame

In [219…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
df.dropna(axis=0, how="any", inplace=True)
print(df)
```

```
   ID   Name  Marks       City
0   1   Amit   78.0  Ahmedabad
1   2  Rahul   85.0      Surat
2   2  Rahul   85.0      Surat
4   4  Priya   90.0     Rajkot
5   4  Priya   90.0     Rajkot
```

Drop Rows with Any NaN (Default)

Drops rows 0,1,2,3 except fully complete rows

Default → axis=0, how="any"

In [222…

```
import pandas as pd
import numpy as np
```

```
df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})

print(df)
df.dropna(inplace=True)
print(df)
```

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
Empty DataFrame
Columns: [Maths, Science, English]
Index: []
```

Drop Rows Only If All Values Are NaN

✔ Removes row 3 onlym

In [223…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})

print(df)
df.dropna(inplace=True,how="all")
print(df)
```

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
```

Drop Columns with Any NaN

removes columns having any NaN

In [226…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})
```

Vishal Acharya

```
print(df)
df.dropna(axis=1, inplace=True)
print(df)
```

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
Empty DataFrame
Columns: []
Index: [0, 1, 2, 3]
```

Drop Columns Only If All NaN

Removes only fully empty columns

In [227…

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})

print(df)
df.dropna(axis=1, how="all",inplace=True)
print(df)
```

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
```

Use thresh

Keeps rows having at least 2 valid values(not NONE value)

In [228…

```
import numpy as np

df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})

print(df)
df.dropna(thresh=2,inplace=True)
print(df)
```

Vishal Acharya

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
```

Drops rows where Maths or Science is NaN

In [229…

```python
import numpy as np

df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})

print(df)
df.dropna(subset=["Maths", "Science"],inplace=True)
print(df)
```

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
   Maths  Science  English
2   90.0     82.0      NaN
```

In [231…

```python
import numpy as np

df = pd.DataFrame({
    "Maths": [85, np.nan, 90, np.nan],
    "Science": [np.nan, 78, 82, np.nan],
    "English": [70, 75, np.nan, np.nan]
})

print(df)
df.dropna(subset="Maths",inplace=True)
print(df)
```

```
   Maths  Science  English
0   85.0      NaN     70.0
1    NaN     78.0     75.0
2   90.0     82.0      NaN
3    NaN      NaN      NaN
   Maths  Science  English
0   85.0      NaN     70.0
2   90.0     82.0      NaN
```

1. **dropna() permanently removes data**

2. Use carefully – data loss possible
3. subset applies only to rows
4. Cannot use how and thresh together
5. Default behavior removes rows with any NaN

Vishal Acharya

In [130…

```python
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "ID": [1, 2, 2, 3, 4, 4],
    "Name": ["Amit", "Rahul", "Rahul", "Neha", "Priya", "Priya"],
    "Marks": [78, 85, 85, np.nan, 90, 90],
    "City": ["Ahmedabad", "Surat", "Surat", "Baroda", "Rajkot", "Rajkot"]
})
```

drop() Method in Pandas DataFrame

Definition: drop() is used to remove rows or columns from a DataFrame based on labels (index/column names).

- It does not modify the original DataFrame by default unless inplace=True.

df.drop(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors="raise")

Parameter	Meaning
`labels`	Row/column labels to remove
`axis=0`	Drop rows
`axis=1`	Drop columns
`index`	Drop rows by index labels
`columns`	Drop columns by column names
`level`	Level name/index in MultiIndex
`inplace=False`	Modify original DataFrame
`errors="raise"`	Raise error if label not found
`errors="ignore"`	Ignore missing labels

Feature	drop()	del
Remove rows	✔	❌
Remove columns	✔	✔
Multiple columns	✔	❌
Inplace control	✔	✔

Drop a Row by Index Label

In [299…

```python
import pandas as pd

df1 = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Marks": [80, 85, 90],
    "Age": [20, 21, 22]
```

Vishal Acharya

```
})

result = df1.drop(1)
print(result)
```

```
  Name  Marks  Age
0    A     80   20
2    C     90   22
```

Drop Multiple Rows

In [300…

```
df2 = pd.DataFrame({
    "Product": ["Pen", "Book", "Bag", "Box"],
    "Price": [10, 50, 30, 20],
    "Stock": [100, 50, 75, 40]
})

result = df2.drop([0, 2])
print(result)
```

```
  Product  Price  Stock
1    Book     50     50
3     Box     20     40
```

Drop Column Using axis=1

In [301…

```
df3 = pd.DataFrame({
    "RollNo": [1, 2, 3],
    "Name": ["A", "B", "C"],
    "Marks": [75, 85, 90]
})

result = df3.drop("Marks", axis=1)
print(result)
```

```
   RollNo Name
0       1    A
1       2    B
2       3    C
```

Drop Multiple Columns

In [302…

```
df4 = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["X", "Y", "Z"],
    "Dept": ["IT", "HR", "Sales"],
    "Salary": [50000, 45000, 60000]
})

result = df4.drop(["Dept", "Salary"], axis=1)
print(result)
```

```
    ID Name
0  101    X
1  102    Y
2  103    Z
```

Drop Rows Using index Parameter

In [303…

```
df5 = pd.DataFrame({
    "City": ["Delhi", "Mumbai", "Chennai"],
```

Vishal Acharya

```
    "Population": [20, 18, 7],
    "State": ["DL", "MH", "TN"]
})

result = df5.drop(index=[0, 2])
print(result)
```

```
    City  Population State
1  Mumbai          18    MH
```

Drop Columns Using columns Parameter

Recommended & clear way to drop columns

In [305…

```
df6 = pd.DataFrame({
    "Student": ["A", "B", "C"],
    "Maths": [80, 85, 90],
    "Science": [78, 82, 88]
})

result = df6.drop(columns=["Science"])
print(result)
```

```
  Student  Maths
0       A     80
1       B     85
2       C     90
```

Drop with inplace=True

In [306…

```
df7 = pd.DataFrame({
    "Emp": ["X", "Y", "Z"],
    "Dept": ["IT", "HR", "Sales"],
    "Salary": [50000, 45000, 60000]
})

df7.drop("Dept", axis=1, inplace=True)
print(df7)
```

```
  Emp  Salary
0   X   50000
1   Y   45000
2   Z   60000
```

Drop with errors="ignore"

No error raised if label not found

In [307…

```
df8 = pd.DataFrame({
    "A": [1, 2, 3],
    "B": [4, 5, 6],
    "C": [7, 8, 9]
})

result = df8.drop("D", axis=1, errors="ignore")
print(result)
```

Vishal Acharya

```
   A  B  C
0  1  4  7
1  2  5  8
2  3  6  9
```

Drop Using level (MultiIndex)

Drops all rows where Letter = "A"

In [308…

```python
arrays = [["A", "A", "B"], [1, 2, 1]]
index = pd.MultiIndex.from_arrays(arrays, names=("Letter", "Number"))

df9 = pd.DataFrame({
    "Marks": [80, 85, 90],
    "Age": [20, 21, 22]
}, index=index)
print(df9)
result = df9.drop("A", level="Letter")
print(result)
```

```
               Marks  Age
Letter Number
A      1          80   20
       2          85   21
B      1          90   22
               Marks  Age
Letter Number
B      1          90   22
```

duplicated()

Definition:is used to identify (mark) duplicate rows in a DataFrame or duplicate values in a Series.

It returns a Boolean Series:

True → duplicate row/value

False → unique row/value

❗ It does not remove duplicates — it only marks them.

📄 Syntax df.duplicated(subset=None, keep="first")

subset

Value	Meaning
`None` (default)	Check duplicates using **all columns**
Column name	Check duplicates based on **one column**
List of columns	Check duplicates using **multiple columns**

KEEP

Value	Meaning
`"first"` (default)	First occurrence → `False`, others → `True`

Vishal Acharya

Value	Meaning
`"last"`	Last occurrence → `False`, others → `True`
`False`	**All duplicates** marked as `True`

Parameter	Meaning
`subset`	Column(s) to check
`keep="first"`	First occurrence unique
`keep="last"`	Last occurrence unique
`keep=False`	All duplicates marked

Vishal Acharya

(keep="first")

In [233…

```
import pandas as pd

df1 = pd.DataFrame({
    "RollNo": [1, 2, 2, 3],
    "Name": ["A", "B", "B", "C"],
    "Marks": [80, 85, 85, 90]
})

print(df1)
df1.duplicated()
```

```
   RollNo Name  Marks
0       1    A     80
1       2    B     85
2       2    B     85
3       3    C     90
```

Out[233…

```
0    False
1    False
2     True
3    False
dtype: bool
```

Mark All Duplicates (keep=False)

All duplicate rows marked True.

In [234…

```
df2 = pd.DataFrame({
    "EmpID": [101, 102, 101, 103],
    "Dept": ["IT", "HR", "IT", "Sales"],
    "Salary": [50000, 45000, 50000, 60000]
})

print(df2)
df2.duplicated(keep=False)
```

```
   EmpID   Dept  Salary
0    101     IT   50000
1    102     HR   45000
2    101     IT   50000
3    103  Sales   60000
```

Out[234…

```
0     True
1    False
2     True
3    False
dtype: bool
```

Keep Last Occurrence (keep="last")

Last duplicate remains False.

In [235…

```
df3 = pd.DataFrame({
    "ProductID": [1, 2, 1, 3],
    "Product": ["Pen", "Pencil", "Pen", "Eraser"],
    "Price": [10, 5, 10, 3]
})

print(df3)
df3.duplicated(keep="last")
```

```
   ProductID Product  Price
0          1     Pen     10
1          2  Pencil      5
2          1     Pen     10
3          3  Eraser      3
```

Out[235…

```
0     True
1    False
2    False
3    False
dtype: bool
```

subset with Single Column

Only StudentID is checked.

In [236…

```
df4 = pd.DataFrame({
    "StudentID": [11, 12, 12, 13],
    "Subject": ["Math", "Sci", "Eng", "Math"],
    "Marks": [78, 82, 85, 90]
})

print(df4)
df4.duplicated(subset="StudentID")
```

```
   StudentID Subject  Marks
0         11    Math     78
1         12     Sci     82
2         12     Eng     85
3         13    Math     90
```

Out[236…

```
0    False
1    False
2     True
3    False
dtype: bool
```

subset with Multiple Columns

Duplicate only when both columns match.

Vishal Acharya

In [237…

```
df5 = pd.DataFrame({
    "Name": ["A", "A", "B", "B"],
    "Course": ["Python", "Python", "Java", "Python"],
    "Duration": [30, 30, 45, 30]
})

print(df5)
df5.duplicated(subset=["Name", "Course"])
```

```
  Name  Course  Duration
0    A  Python        30
1    A  Python        30
2    B    Java        45
3    B  Python        30
```

Out[237…

```
0    False
1     True
2    False
3    False
dtype: bool
```

filter Only Duplicate Rows

Shows only duplicate rows.

In [238…

```
df6 = pd.DataFrame({
    "Email": ["a@gmail.com", "b@gmail.com", "a@gmail.com"],
    "User": ["X", "Y", "X"],
    "Status": ["Active", "Active", "Active"]
})

print(df6)
df6[df6.duplicated()]
```

```
        Email User  Status
0  a@gmail.com    X  Active
1  b@gmail.com    Y  Active
2  a@gmail.com    X  Active
```

Out[238…

	Email	User	Status
2	a@gmail.com	X	Active

Count Duplicate Rows

Counts number of duplicate rows.

In [240…

```
df7 = pd.DataFrame({
    "OrderID": [1, 2, 2, 3],
    "Item": ["Pen", "Book", "Book", "Bag"],
    "Qty": [2, 1, 1, 3]
})

print(df7)
df7.duplicated().sum()
```

Vishal Acharya

```
   OrderID  Item  Qty
0        1   Pen    2
1        2  Book    1
2        2  Book    1
3        3   Bag    3
```

Out[240… 1

drop_duplicates() in Pandas

Definition: drop_duplicates() is used to remove duplicate rows from a DataFrame or Series and keep only unique records based on given rules.

Syntax: df.drop_duplicates(subset=None, keep="first", inplace=False, ignore_index=False)

Parameters

1. subset

Value	Meaning
`None` (default)	Consider **all columns**
Column name	Remove duplicates based on **one column**
List of columns	Remove duplicates based on **multiple columns**

2. keep

Value	Meaning
`"first"` (default)	Keep **first occurrence**
`"last"`	Keep **last occurrence**
`False`	Remove **all duplicates**

3. inplace

Value	Meaning
`True`	Modify original DataFrame
`False`	Return new DataFrame

4.

Value	Meaning
`True`	Reset index after removing duplicates
`False`	Keep original index

Feature	duplicated()	drop_duplicates()
Purpose	Mark duplicates	Remove duplicates
Output	Boolean Series	DataFrame

Vishal Acharya

Feature	duplicated()	drop_duplicates()
Data loss	❌ No	✔ Yes

Important Exam Notes

1. Default: keep="first"
2. subset works only for DataFrame
3. ignore_index=True resets index
4. keep=False removes all duplicates
5. Use carefully → data loss possible

Default (keep="first")

Keeps first occurrence of duplicate rows.

In [241…

```
import pandas as pd

df1 = pd.DataFrame({
    "RollNo": [1, 2, 2, 3],
    "Name": ["A", "B", "B", "C"],
    "Marks": [80, 85, 85, 90]
})

result = df1.drop_duplicates()
print(result)
```

```
   RollNo Name  Marks
0       1    A     80
1       2    B     85
3       3    C     90
```

Keep Last Duplicate

Keeps last repeated row.

In [242…

```
df2 = pd.DataFrame({
    "EmpID": [101, 102, 101, 103],
    "Dept": ["IT", "HR", "IT", "Sales"],
    "Salary": [50000, 45000, 50000, 60000]
})

result = df2.drop_duplicates(keep="last")
print(result)
```

```
   EmpID   Dept  Salary
1    102     HR   45000
2    101     IT   50000
3    103  Sales   60000
```

Remove All Duplicates (keep=False)

Completely removes all duplicate rows.

In [243…

```
df3 = pd.DataFrame({
    "City": ["Delhi", "Mumbai", "Delhi", "Chennai"],
    "State": ["DL", "MH", "DL", "TN"],
```

Vishal Acharya

```
    "Pin": [110001, 400001, 110001, 600001]
})

result = df3.drop_duplicates(keep=False)
print(result)
```

```
     City State     Pin
1   Mumbai    MH  400001
3  Chennai    TN  600001
```

subset with Single Column

Duplicate check only on StudentID.

In [244…

```
df4 = pd.DataFrame({
    "StudentID": [11, 12, 12, 13],
    "Subject": ["Math", "Sci", "Eng", "Math"],
    "Marks": [78, 82, 85, 90]
})

result = df4.drop_duplicates(subset="StudentID")
print(result)
```

```
   StudentID Subject  Marks
0         11    Math     78
1         12     Sci     82
3         13    Math     90
```

Subset with Multiple Columns

Duplicate only if Name + Course both match.

In [245…

```
df5 = pd.DataFrame({
    "Name": ["A", "A", "B", "B"],
    "Course": ["Python", "Python", "Java", "Python"],
    "Duration": [30, 30, 45, 30]
})

result = df5.drop_duplicates(subset=["Name", "Course"])
print(result)
```

```
  Name  Course  Duration
0    A  Python        30
2    B    Java        45
3    B  Python        30
```

ignore_index=True

Index reset after removing duplicates.

In [246…

```
df6 = pd.DataFrame({
    "OrderID": [1, 2, 2, 3],
    "Item": ["Pen", "Book", "Book", "Bag"],
    "Qty": [2, 1, 1, 3]
})

result = df6.drop_duplicates(ignore_index=True)
print(result)
```

Vishal Acharya

```
   OrderID  Item  Qty
0        1   Pen    2
1        2  Book    1
2        3   Bag    3
```

inplace=True

original DataFrame is permanently changed

In [247…

```python
df7 = pd.DataFrame({
    "Email": ["a@gmail.com", "b@gmail.com", "a@gmail.com"],
    "User": ["X", "Y", "X"],
    "Status": ["Active", "Active", "Active"]
})

df7.drop_duplicates(inplace=True)
print(df7)
```

```
        Email User  Status
0  a@gmail.com    X  Active
1  b@gmail.com    Y  Active
```

SORTING METHODS

sort_values() in Pandas

Definition: sort_values() is used to sort a DataFrame or Series based on column values in ascending or descending order.

df.sort_values(by, axis=0, ascending=True, inplace=False, kind="quicksort", na_position="last", ignore_index=False)

Parameter	Meaning
`by`	Column name or list of column names to sort
`axis=0`	Sort rows (default)
`ascending=True`	Sort order (`True` = ascending, `False` = descending)
`inplace=False`	Modify original DataFrame
`kind`	Sorting algorithm (`quicksort` , `mergesort` , `heapsort`)
`na_position="last"`	Position of NaN values (`first` / `last`)
`ignore_index=False`	Reset index after sorting

Feature	sort_values()	sort_index()
Sort by	Column values	Index
Common use	Data analysis	Index management

Important Exam Notes

1. by parameter is mandatory
2. Multiple columns allowed

Vishal Acharya

3. NaN handling using na_position
4. ignore_index=True resets index
5. Default order → ascending

Sort by Single Column (Ascending – Default)

Sorts rows by Marks (low → high).

In [248…

```
import pandas as pd

df1 = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Marks": [78, 85, 72],
    "Age": [20, 21, 19]
})

result = df1.sort_values(by="Marks")
print(result)
```

```
  Name  Marks  Age
2    C     72   19
0    A     78   20
1    B     85   21
```

Sort by Single Column (Descending)

Highest salary comes first.

In [249…

```
df2 = pd.DataFrame({
    "Employee": ["X", "Y", "Z"],
    "Salary": [50000, 60000, 45000],
    "Experience": [3, 5, 2]
})

result = df2.sort_values(by="Salary", ascending=False)
print(result)
```

```
  Employee  Salary  Experience
1        Y   60000           5
0        X   50000           3
2        Z   45000           2
```

Sort by Multiple Columns

First sorted by Dept, then by Salary.

In [250…

```
df3 = pd.DataFrame({
    "Dept": ["IT", "IT", "HR", "HR"],
    "Name": ["A", "B", "C", "D"],
    "Salary": [50000, 60000, 50000, 45000]
})

result = df3.sort_values(by=["Dept", "Salary"])
print(result)
```

Vishal Acharya

```
  Dept Name  Salary
3   HR    D   45000
2   HR    C   50000
0   IT    A   50000
1   IT    B   60000
```

Different Order for Each Column

Class → ascending

Marks → descending

In [251…

```
df4 = pd.DataFrame({
    "Class": [10, 10, 9, 9],
    "Marks": [85, 75, 90, 70],
    "RollNo": [2, 1, 4, 3]
})

result = df4.sort_values(
    by=["Class", "Marks"],
    ascending=[True, False]
)
print(result)
```

```
   Class  Marks  RollNo
2      9     90       4
3      9     70       3
0     10     85       2
1     10     75       1
```

Sorting with NaN Values

NaN values appear at the top.

In [252…

```
df5 = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Score": [90, None, 85],
    "Rank": [1, 3, 2]
})

result = df5.sort_values(by="Score", na_position="first")
print(result)
```

```
  Name  Score  Rank
1    B    NaN     3
2    C   85.0     2
0    A   90.0     1
```

Reset Index After Sorting

Index reset after sorting.

In [253…

```
df6 = pd.DataFrame({
    "Product": ["Pen", "Book", "Bag"],
    "Price": [10, 50, 30],
    "Stock": [100, 50, 75]
})
```

Vishal Acharya

```
result = df6.sort_values(by="Price", ignore_index=True)
print(result)
```

```
  Product  Price  Stock
0     Pen     10    100
1     Bag     30     75
2    Book     50     50
```

Modify Original DataFrame (inplace=True)

In [254…

```
df7 = pd.DataFrame({
    "Student": ["A", "B", "C"],
    "Maths": [65, 85, 75],
    "Science": [70, 80, 78]
})

df7.sort_values(by="Maths", inplace=True)
print(df7)
```

```
  Student  Maths  Science
0       A     65       70
2       C     75       78
1       B     85       80
```

sort_index() in Pandas

Definition: sort_index() is used to sort a DataFrame or Series based on its index values instead of column values.

df.sort_index(axis=0, level=None, ascending=True, inplace=False, kind="quicksort", na_position="last", ignore_index=False)

Parameter	Meaning
`axis=0`	Sort by row index (1 = column index)
`level`	Sort by specific level (MultiIndex)
`ascending=True`	Sort order
`inplace=False`	Modify original DataFrame
`kind`	Sorting algorithm
`na_position="last"`	Position of NaN index
`ignore_index=False`	Reset index after sorting

Important Exam Notes

1. Default → row index sorting
2. axis=1 sorts columns
3. ignore_index=True resets index
4. level used for MultiIndex
5. NaN handled using na_position

Default Index Sorting (Ascending)

Vishal Acharya

In [255…

```python
import pandas as pd

df1 = pd.DataFrame(
    {
        "Name": ["A", "B", "C"],
        "Marks": [85, 78, 90],
        "Age": [20, 21, 19]
    },
    index=[3, 1, 2]
)

result = df1.sort_index()
print(result)
```

```
  Name  Marks  Age
1    B     78   21
2    C     90   19
3    A     85   20
```

Descending Index Order

Index sorted high → low.

In [256…

```python
df2 = pd.DataFrame(
    {
        "Product": ["Pen", "Book", "Bag"],
        "Price": [10, 50, 30],
        "Stock": [100, 50, 75]
    },
    index=[101, 103, 102]
)

result = df2.sort_index(ascending=False)
print(result)
```

```
    Product  Price  Stock
103    Book     50     50
102     Bag     30     75
101     Pen     10    100
```

Sort Columns by Column Index (axis=1)

Columns sorted alphabetically.

In [257…

```python
df3 = pd.DataFrame({
    "C": [1, 2],
    "A": [3, 4],
    "B": [5, 6]
})

result = df3.sort_index(axis=1)
print(result)
```

```
   A  B  C
0  3  5  1
1  4  6  2
```

Sorting with String Index

Index sorted alphabetically (a → c).

Vishal Acharya

In [258…

```
df4 = pd.DataFrame(
    {
        "City": ["Delhi", "Mumbai", "Chennai"],
        "Population": [20, 18, 7],
        "State": ["DL", "MH", "TN"]
    },
    index=["b", "c", "a"]
)

result = df4.sort_index()
print(result)
```

```
      City  Population State
a  Chennai           7    TN
b    Delhi          20    DL
c   Mumbai          18    MH
```

Sorting with NaN Index Values

NaN index placed at the top.

In [259…

```
df5 = pd.DataFrame(
    {
        "Item": ["Pen", "Book", "Bag"],
        "Qty": [10, 5, 8],
        "Price": [20, 50, 30]
    },
    index=[2, None, 1]
)

result = df5.sort_index(na_position="first")
print(result)
```

```
     Item  Qty  Price
NaN  Book    5     50
1.0   Bag    8     30
2.0   Pen   10     20
```

Reset Index After Sorting

Index reset to 0,1,2.

In [261…

```
df6 = pd.DataFrame(
    {
        "Student": ["A", "B", "C"],
        "Maths": [65, 85, 75],
        "Science": [70, 80, 78]
    },
    index=[5, 3, 4]
)

result = df6.sort_index(ignore_index=True)
print(result)
```

```
  Student  Maths  Science
0       B     85       80
1       C     75       78
2       A     65       70
```

Modify Original DataFrame (inplace=True)

Vishal Acharya

In [262…

```
df7 = pd.DataFrame(
    {
        "Emp": ["X", "Y", "Z"],
        "Dept": ["IT", "HR", "Sales"],
        "Salary": [50000, 45000, 60000]
    },
    index=[2, 3, 1]
)

df7.sort_index(inplace=True)
print(df7)
```

```
  Emp   Dept  Salary
1   Z  Sales   60000
2   X     IT   50000
3   Y     HR   45000
```

Feature	sort_index()	sort_values()
Sort by	Index	Column values
Use case	Index ordering	Data ordering

Statistical Methods in Pandas (DataFrame)

Statistical methods are used to analyze numerical data by calculating totals, averages, minimum, maximum, variation, etc. axis parameter use for row,column wise operation

Method	Definition
`sum()`	Returns the total of values
`mean()`	Returns the average value
`median()`	Returns the middle value
`min()`	Returns the smallest value
`max()`	Returns the largest value
`mode()`	Returns the most frequent value
`count()`	Counts non-null values
`std()`	Measures data spread from mean
`var()`	Variance (square of std)
`quantile()`	Returns value at given percentile
`corr()`	Correlation between columns
`describe()`	Summary of statistics
`idxmax()`	Index of maximum value
`idxmin()`	Index of minimum value

In [3]:

```
import pandas as pd
```

Vishal Acharya

```python
# Create DataFrame
df = pd.DataFrame({
    "Maths": [80, 85, 90, 75, 85],
    "Science": [78, 82, 88, 80, 82],
    "English": [70, 75, 80, 72, 75]
})

print("DataFrame:")
print(df)

print("\nSUM:")
print(df.sum())
print(df.sum(axis=1))

print("\nMEAN:")
print(df.mean())
print(df.mean(axis=1))

print("\nMEDIAN:")
print(df.median())

print("\nMIN:")
print(df.min())

print("\nMAX:")
print(df.max())

print("\nMODE:")
print(df.mode())

print("\nCOUNT (Non-NaN values):")
print(df.count())

print("\nSTANDARD DEVIATION:")
print(df.std())

print("\nVARIANCE:")
print(df.var())

print("\n25th Quantile:")
print(df.quantile(0.25))

print("\nCORRELATION:")
print(df.corr())

print("\nDESCRIBE:")
print(df.describe())

print("\nINDEX OF MAX VALUE:")
print(df.idxmax())

print("\nINDEX OF MIN VALUE:")
print(df.idxmin())
```

Vishal Acharya

```
DataFrame:
   Maths  Science  English
0     80       78       70
1     85       82       75
2     90       88       80
3     75       80       72
4     85       82       75

SUM:
Maths      415
Science    410
English    372
dtype: int64
0    228
1    242
2    258
3    227
4    242
dtype: int64

MEAN:
Maths      83.0
Science    82.0
English    74.4
dtype: float64
0    76.000000
1    80.666667
2    86.000000
3    75.666667
4    80.666667
dtype: float64

MEDIAN:
Maths      85.0
Science    82.0
English    75.0
dtype: float64

MIN:
Maths      75
Science    78
English    70
dtype: int64

MAX:
Maths      90
Science    88
English    80
dtype: int64

MODE:
   Maths  Science  English
0     85       82       75

COUNT (Non-NaN values):
Maths      5
Science    5
English    5
dtype: int64
```

Vishal Acharya

```
STANDARD DEVIATION:
Maths      5.700877
Science    3.741657
English    3.781534
dtype: float64

VARIANCE:
Maths      32.5
Science    14.0
English    14.3
dtype: float64

25th Quantile:
Maths      80.0
Science    80.0
English    72.0
Name: 0.25, dtype: float64

CORRELATION:
            Maths   Science   English
Maths    1.000000  0.820413  0.858148
Science  0.820413  1.000000  0.989455
English  0.858148  0.989455  1.000000

DESCRIBE:
           Maths    Science    English
count   5.000000   5.000000   5.000000
mean   83.000000  82.000000  74.400000
std     5.700877   3.741657   3.781534
min    75.000000  78.000000  70.000000
25%    80.000000  80.000000  72.000000
50%    85.000000  82.000000  75.000000
75%    85.000000  82.000000  75.000000
max    90.000000  88.000000  80.000000

INDEX OF MAX VALUE:
Maths      2
Science    2
English    2
dtype: int64

INDEX OF MIN VALUE:
Maths      3
Science    0
English    0
dtype: int64
```

df.corr() in Pandas

Definition: df.corr() computes the pairwise correlation between numerical columns of a DataFrame. Correlation shows how strongly two variables are related.

Correlation value range:

+1 → Perfect positive relationship

0 → No relationship

Vishal Acharya

-1 → Perfect negative relationship

Syntax: df.corr(method="pearson", min_periods=1, numeric_only=False)

1. method

Value	Meaning	When to Use
`"pearson"` (default)	Linear correlation	Continuous, normally distributed data
`"spearman"`	Rank correlation	Non-linear but monotonic data
`"kendall"`	Ordinal association	Small datasets, rankings

2. min_periods: Minimum number of non-NaN values required to calculate correlation

If valid values are less than this → result is NaN

3. numeric_only: Useful when DataFrame has mixed data types

Value	Meaning
`False` (default)	Include all numeric columns automatically
`True`	Explicitly use numeric columns only

Value	Interpretation
`> 0.7`	Strong positive
`0.3 – 0.7`	Moderate
`< 0.3`	Weak
`< 0`	Negative

Important Exam Notes

1. Works only on numeric data
2. NaN values are ignored automatically
3. Output is a square matrix
4. Diagonal values always = 1.0
5. Correlation ≠ causation

Default Correlation (Pearson)

Shows linear relationship between subjects.

In [264…

```
import pandas as pd

df1 = pd.DataFrame({
    "Maths": [80, 85, 90, 75, 85],
    "Science": [78, 82, 88, 80, 82],
    "English": [70, 75, 80, 72, 75]
})

print(df1.corr())
```

Vishal Acharya

```
            Maths   Science   English
Maths    1.000000  0.820413  0.858148
Science  0.820413  1.000000  0.989455
English  0.858148  0.989455  1.000000
```

Spearman Correlation

Uses ranking instead of raw values.

In [265…

```python
df2 = pd.DataFrame({
    "Rank": [1, 2, 3, 4, 5],
    "Marks": [90, 85, 80, 78, 70],
    "Age": [18, 19, 20, 21, 22]
})

print(df2.corr(method="spearman"))
```

```
      Rank  Marks  Age
Rank   1.0   -1.0  1.0
Marks -1.0    1.0 -1.0
Age    1.0   -1.0  1.0
```

Kendall Correlation

Best for ordinal / rank-based data.

In [266…

```python
df3 = pd.DataFrame({
    "Judge1": [8, 7, 9, 6],
    "Judge2": [7, 6, 8, 5],
    "Judge3": [9, 8, 10, 7]
})

print(df3.corr(method="kendall"))
```

```
        Judge1  Judge2  Judge3
Judge1     1.0     1.0     1.0
Judge2     1.0     1.0     1.0
Judge3     1.0     1.0     1.0
```

Using min_periods

Correlation calculated only if at least 3 valid values exist.

In [267…

```python
df4 = pd.DataFrame({
    "A": [1, 2, None, 4],
    "B": [2, None, 6, 8],
    "C": [1, 3, 5, 7]
})

print(df4.corr(min_periods=3))
```

```
     A    B    C
A  1.0  NaN  1.0
B  NaN  1.0  1.0
C  1.0  1.0  1.0
```

numeric_only=True

Ignores non-numeric columns automatically.

Vishal Acharya

In [268…

```
df5 = pd.DataFrame({
    "Name": ["A", "B", "C", "D"],
    "Maths": [80, 85, 90, 88],
    "Science": [78, 82, 88, 85]
})

print(df5.corr(numeric_only=True))
```

```
           Maths   Science
Maths    1.000000  0.991191
Science  0.991191  1.000000
```

Correlation of Two Specific Columns

Correlation between two Series only.

In [269…

```
df6 = pd.DataFrame({
    "Height": [150, 160, 170, 180],
    "Weight": [50, 60, 70, 80],
    "Age": [20, 22, 24, 26]
})

print(df6["Height"].corr(df6["Weight"]))
```

```
1.0
```

Scatter Matrix in Pandas

A scatter matrix is a grid of plots where:

Diagonal → distribution (histogram or KDE) of each variable

Off-diagonal → scatter plots between every pair of numerical columns

It is mainly used to identify relationships, trends, and correlations between multiple variables.

pd.plotting.scatter_matrix(frame, alpha=0.5, figsize=None, ax=None, grid=False, diagonal="hist", marker=".", density_kwds=None, hist_kwds=None, range_padding=0.05)

Parameter	Meaning
`frame`	DataFrame containing numerical data
`alpha`	Transparency of points (0–1)
`figsize`	Size of the figure (width, height)
`ax`	Matplotlib axes object
`grid`	Show grid lines (`True/False`)
`diagonal`	Plot type on diagonal (`"hist"` or `"kde"`)
`marker`	Marker style for scatter plots
`density_kwds`	Extra arguments for KDE plot

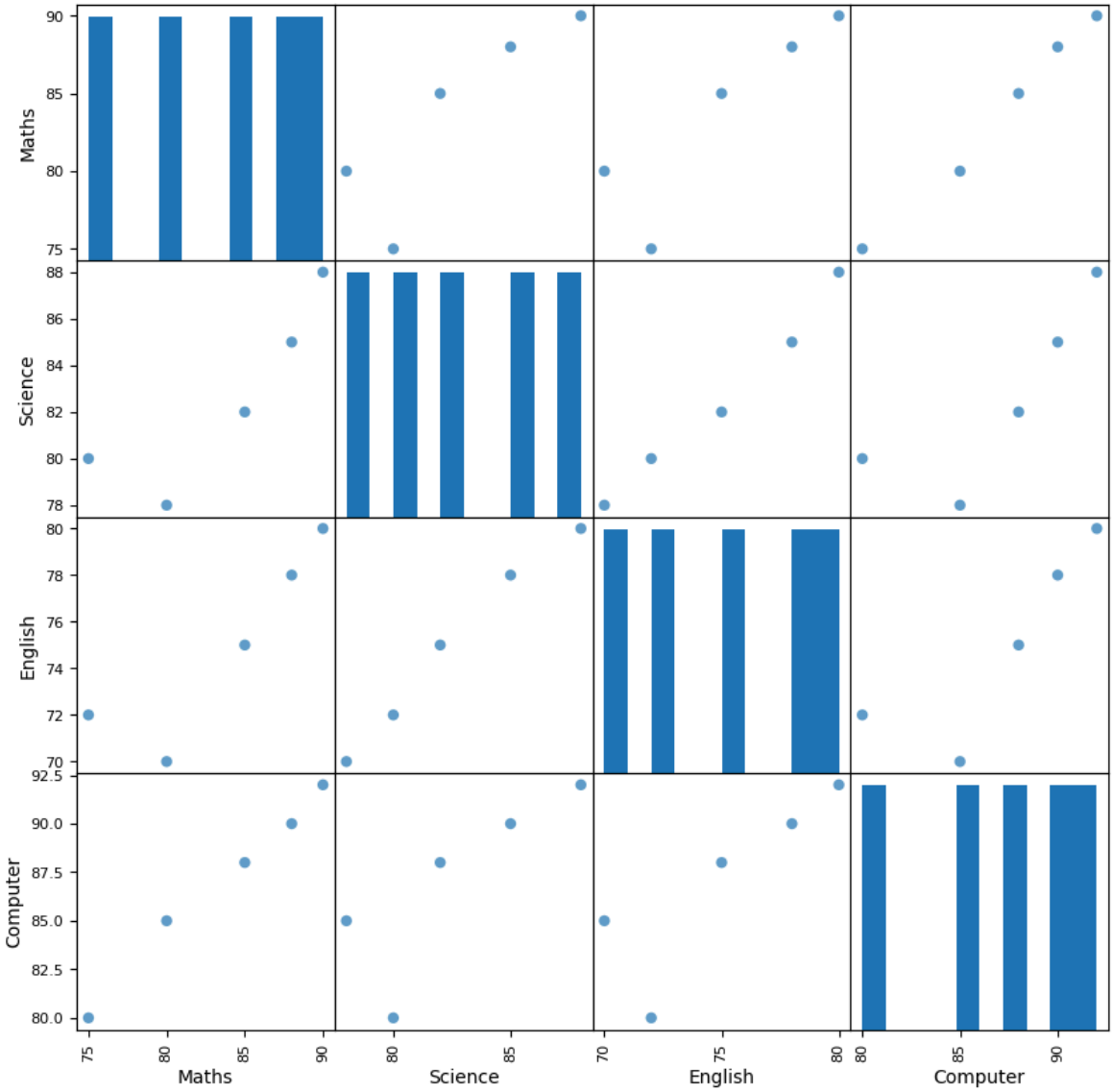
Vishal Acharya

Parameter	Meaning
`hist_kwds`	Extra arguments for histogram
`range_padding`	Extra spacing around plot range

In [7]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
df = pd.DataFrame({
    "Maths": [80, 85, 90, 75, 88],
    "Science": [78, 82, 88, 80, 85],
    "English": [70, 75, 80, 72, 78],
    "Computer": [85, 88, 92, 80, 90]
})

# Scatter Matrix
pd.plotting.scatter_matrix(
    df,
    alpha=0.7,                  # transparency
    figsize=(10, 10),           # figure size
    grid=True,                  # show grid
    diagonal="hist",            # histogram on diagonal
    marker="o",                 # marker type
    hist_kwds={"bins": 10},     # histogram settings
    range_padding=0.1           # spacing
)

plt.show()
```

Vishal Acharya

Vishal Acharya



ths vs Science, Maths vs English, etc. → scatter plots

Diagonal plots → distribution of each subject

Helps visually detect:

Positive correlation

Negative correlation

Outliers

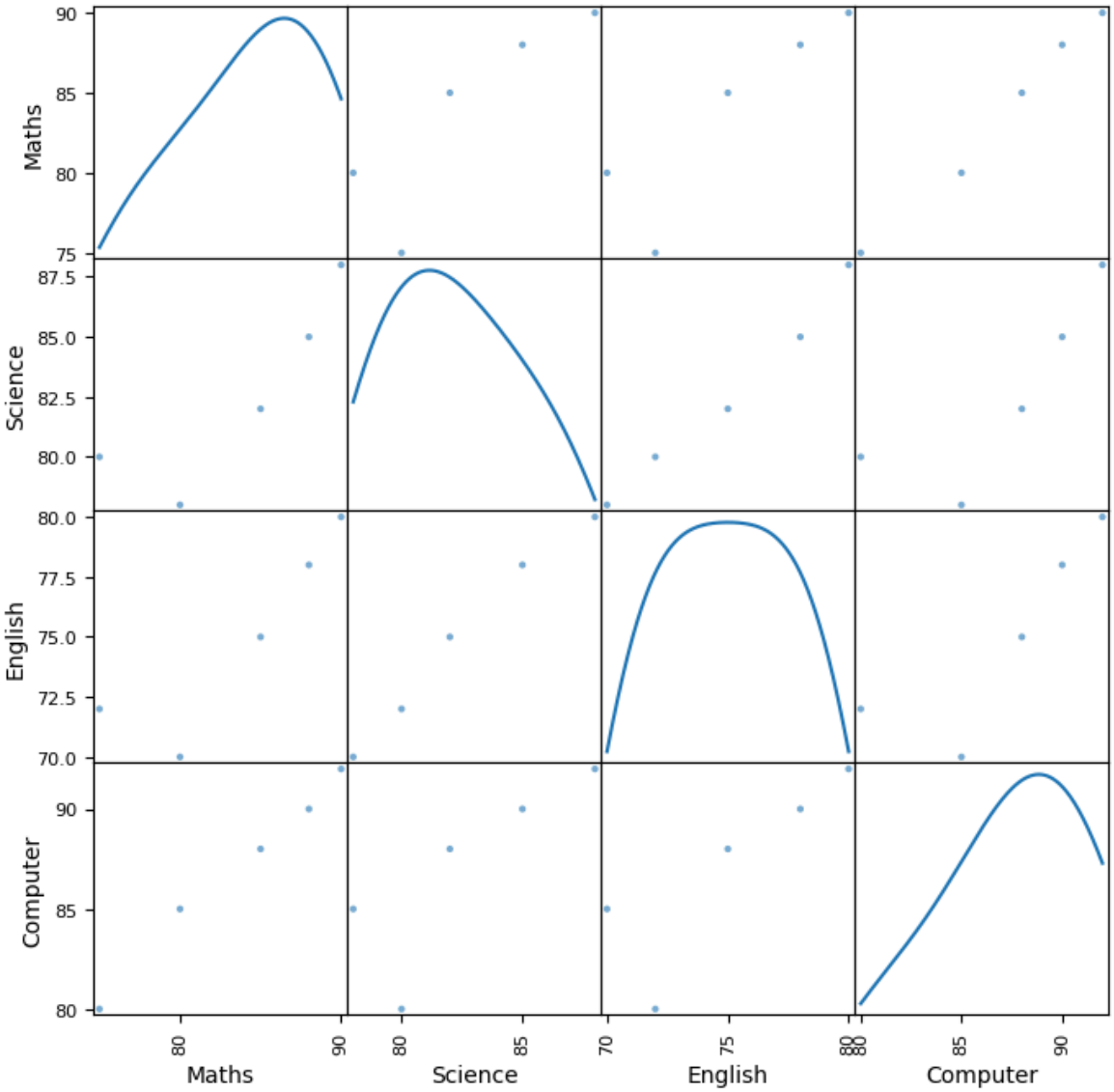
Clusters

KDE gives a smooth density curve instead of bars.

In [271…

```
pd.plotting.scatter_matrix(
    df,
    diagonal="kde",
    alpha=0.6,
    figsize=(8, 8)
)
plt.show()
```

Vishal Acharya



Importan Notes

1. Works only with numeric columns
2. Diagonal always represents self-distribution
3. Scatter matrix is part of EDA (Exploratory Data Analysis)
4. Use alpha to avoid point overlapping
5. Large datasets → plots may become slow

Scatter Matrix	Correlation
Visual	Numerical
Shows patterns	Shows strength
Detects outliers	No outlier info

parallel_coordinates(frame, class_column, cols=None, color=None)

parallel_coordinates() is used to visualize multivariate numerical data in a single plot.

Purpose

It draws a Parallel Coordinates Plot, where:

Each vertical line = one feature (column)

Each horizontal colored line = one row (record)

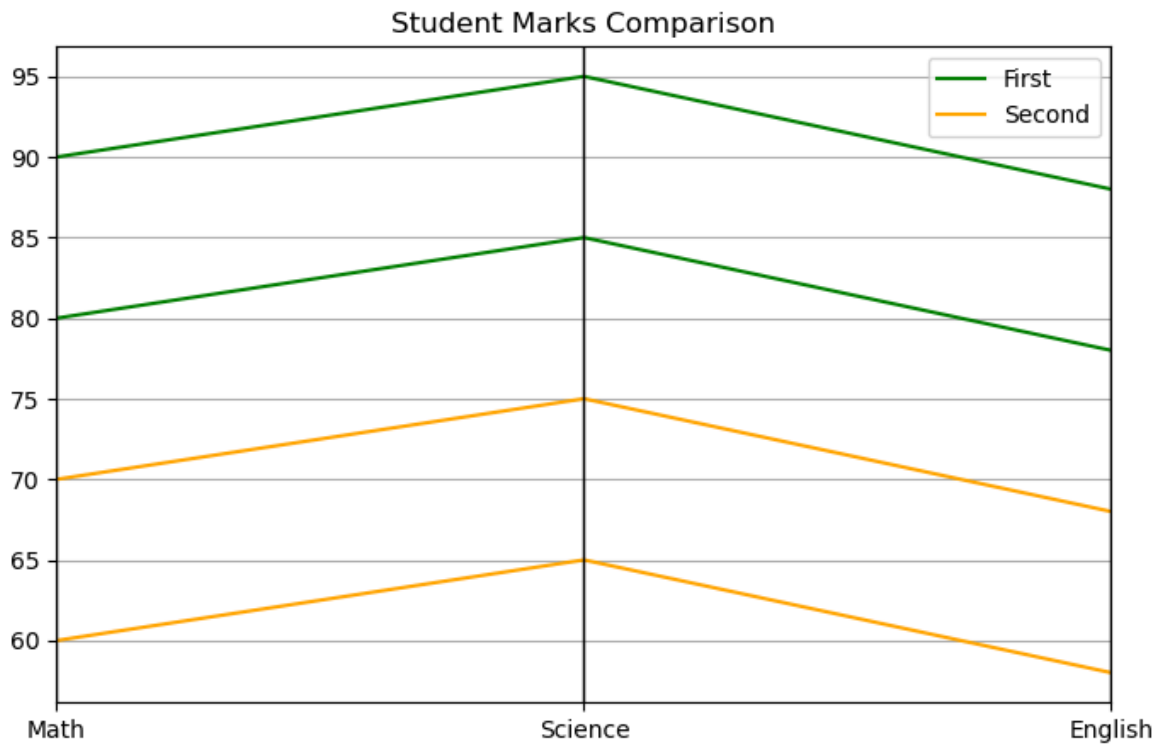
Colors are based on a class/category column

Parameter	Meaning
`frame`	The Pandas DataFrame
`class_column`	Categorical column used for grouping & coloring
`cols`	List of specific columns to plot (default = all numeric columns except class column)
`color`	List of colors for different classes

In [6]:

```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates

# Create Data
df = pd.DataFrame({
    'Math': [80, 70, 90, 60],
    'Science': [85, 75, 95, 65],
    'English': [78, 68, 88, 58],
    'Grade': ['First', 'Second', 'First', 'Second']
})

plt.figure(figsize=(8,5))

parallel_coordinates(
    frame=df,
    class_column='Grade',
    cols=['Math', 'Science', 'English'],
    color=['green', 'orange']
)

plt.title("Student Marks Comparison")
plt.show()
```

Vishal Acharya



pandas.crosstab()

crosstab() in Pandas is used to create a cross-tabulation table (frequency table). It shows the relationship between two or more categorical variables.

pd.crosstab(index, columns,margins=False)

In [7]:
```
import pandas as pd

data = {
    "Gender": ["Male", "Female", "Male", "Female", "Male"],
    "Result": ["Pass", "Pass", "Fail", "Pass", "Fail"]
}

df = pd.DataFrame(data)

table = pd.crosstab(df["Gender"], df["Result"])
print(table)
```

```
Result  Fail  Pass
Gender
Female     0     2
Male       2     1
```

In [8]:
```
pd.crosstab(df["Gender"], df["Result"], margins=True)
```

Vishal Acharya

Out[8]:

Result	Fail	Pass	All
Gender			
Female	0	2	2
Male	2	1	3
All	2	3	5

TRANSFORMATION METHODS

apply(): Apply function row/column wise

apply() is used to apply a function along rows or columns of a DataFrame. It allows you to perform custom operations that are not available as built-in methods.

df.apply(func, axis=0, raw=False, result_type=None, args=(), **kwargs)

Parameter	Meaning
`func`	Function to apply (built-in or user-defined)
`axis=0`	Apply function **column-wise**
`axis=1`	Apply function **row-wise**
`raw=False`	Pass Series (`False`) or NumPy array (`True`)
`result_type`	Output format (`expand` , `reduce` , `broadcast`)
`args`	Extra positional arguments
`**kwargs`	Extra keyword arguments

important Exam Notes

1. axis=0 → column-wise
2. axis=1 → row-wise
3. Prefer vectorized operations when possible
4. apply() is powerful but slower
5. Use lambda for short logic

Feature	apply()	Built-in
Flexibility	High	Limited
Speed	Slower	Faster
Custom logic	✔	✘

Column-wise Apply (axis=0) using sum()

Calculates sum of each column

In [272…

```
import pandas as pd
```

Vishal Acharya

```
df1 = pd.DataFrame({
    "Maths": [80, 85, 90],
    "Science": [78, 82, 88],
    "English": [70, 75, 80]
})

result = df1.apply(sum)
print(result)
```

```
Maths      255
Science    248
English    225
dtype: int64
```

Column-wise Mean using lambda

Calculates mean of each column

In [273…

```
df2 = pd.DataFrame({
    "Physics": [60, 70, 80],
    "Chemistry": [65, 75, 85],
    "Maths": [70, 80, 90]
})

result = df2.apply(lambda x: x.mean())
print(result)
```

```
Physics      70.0
Chemistry    75.0
Maths        80.0
dtype: float64
```

Row-wise Apply (axis=1) – Total Marks

Calculates row-wise total

In [274…

```
df3 = pd.DataFrame({
    "Sub1": [50, 60, 70],
    "Sub2": [55, 65, 75],
    "Sub3": [60, 70, 80]
})

result = df3.apply(lambda x: x.sum(), axis=1)
print(result)
```

```
0    165
1    195
2    225
dtype: int64
```

row-wise Percentage Calculation

Calculates percentage per row

In [275…

```
df4 = pd.DataFrame({
    "Maths": [80, 90, 85],
    "Science": [75, 85, 80],
    "English": [70, 80, 75]
})
```

Vishal Acharya

```
result = df4.apply(lambda x: (x.sum()/300)*100, axis=1)
print(result)
```

```
0    75.0
1    85.0
2    80.0
dtype: float64
```

Apply with User-Defined Function

Uses custom logic with apply()

In [276…

```
df5 = pd.DataFrame({
    "Marks1": [85, 70, 90],
    "Marks2": [80, 65, 95],
    "Marks3": [75, 60, 85]
})

def grade(total):
    if total >= 250:
        return "A"
    elif total >= 200:
        return "B"
    else:
        return "C"

df5["Total"] = df5.apply(lambda x: x.sum(), axis=1)
df5["Grade"] = df5["Total"].apply(grade)

print(df5)
```

```
   Marks1  Marks2  Marks3  Total Grade
0      85      80      75    240     B
1      70      65      60    195     C
2      90      95      85    270     A
```

raw=True (NumPy array passed)

Faster execution using NumPy array

In [4]:

```
df6 = pd.DataFrame({
    "A": [1, 2, 3],
    "B": [4, 5, 6],
    "C": [7, 8, 9]
})

result = df6.apply(sum, axis=1, raw=True)
print(result)
print(type(result))
```

```
0    12
1    15
2    18
dtype: int64
<class 'pandas.core.series.Series'>
```

In [8]:

```
df6 = pd.DataFrame({
    "A": [1, 2, 3],
    "B": [4, 5, 6],
    "C": [7, 8, 9]
})
```

Vishal Acharya

```
result = df6.apply(sum, axis=1, raw=False)
print(result)
print(type(result))
```

```
0    12
1    15
2    18
dtype: int64
<class 'pandas.core.series.Series'>
```

result_type="expand"

Output expanded into multiple columns

In [279…

```
df7 = pd.DataFrame({
    "Marks": [80, 90, 85],
    "Bonus": [5, 10, 5],
    "Penalty": [2, 1, 3]
})

result = df7.apply(
    lambda x: [x.sum(), x.mean()],
    axis=1,
    result_type="expand"
)

print(result)
```

```
       0          1
0   87.0  29.000000
1  101.0  33.666667
2   93.0  31.000000
```

result_type="broadcast"

Output keeps same DataFrame shape

In [280…

```
df8 = pd.DataFrame({
    "X": [10, 20, 30],
    "Y": [15, 25, 35],
    "Z": [20, 30, 40]
})

result = df8.apply(lambda x: x - x.mean(), result_type="broadcast")
print(result)
```

```
    X   Y   Z
0 -10 -10 -10
1   0   0   0
2  10  10  10
```

Apply with Extra Arguments (args)

In [281…

```
df9 = pd.DataFrame({
    "Score1": [50, 60, 70],
    "Score2": [55, 65, 75],
    "Score3": [60, 70, 80]
})
```

Vishal Acharya

```
def add_bonus(x, bonus):
    return x + bonus

result = df9.apply(add_bonus, axis=1, args=(5,))
print(result)
```

```
   Score1  Score2  Score3
0      55      60      65
1      65      70      75
2      75      80      85
```

astype() in Pandas

Definition: astype() is used to change (cast) the data type of a Pandas Series or DataFrame column(s).

It is commonly used for:

Converting string → int / float

int → float

numeric → string

object → category

fixing wrong data types

Syntax: df.astype(dtype, copy=True, errors="raise")

Parameter	Meaning
`dtype`	Target data type (single type or dict)
`copy=True`	Returns a copy (False may modify view)
`errors="raise"`	Raise error on failure
`errors="ignore"`	Ignore error and keep original

Convert Column to Integer

Convert Column to Float

In [288…

```
import pandas as pd

df1 = pd.DataFrame({
    "Age": ["20", "21", "22"],
    "Marks": [80, 85, 90],
    "Grade": ["A", "B", "A"]
})

df1["Age"] = df1["Age"].astype(int)
print(df1.dtypes)

df2 = pd.DataFrame({
    "Price": [10, 20, 30],
    "Discount": ["5.5", "10.0", "7.5"],
    "Stock": [100, 150, 200]
```

Vishal Acharya

```
})

df2["Discount"] = df2["Discount"].astype(float)
print(df2.dtypes)
```

```
Age       int32
Marks     int64
Grade    object
dtype: object
Price         int64
Discount    float64
Stock         int64
dtype: object
```

Convert Multiple Columns Using Dictionary

In [289…

```
df4 = pd.DataFrame({
    "ID": ["101", "102", "103"],
    "Salary": ["50000", "60000", "55000"],
    "Dept": ["IT", "HR", "IT"]
})

df4 = df4.astype({
    "ID": int,
    "Salary": float
})

print(df4.dtypes)
```

```
ID          int32
Salary    float64
Dept       object
dtype: object
```

using errors="ignore"

Invalid value "abc" is ignored

Column remains unchanged

In [290…

```
df7 = pd.DataFrame({
    "Value": ["10", "20", "abc"],
    "Score": [50, 60, 70],
    "Level": ["A", "B", "C"]
})

df7["Value"] = df7["Value"].astype(int, errors="ignore")
print(df7)
```

```
  Value  Score Level
0    10     50     A
1    20     60     B
2   abc     70     C
```

Convert Entire DataFrame

In [291…

```
df8 = pd.DataFrame({
    "A": [1.2, 2.3, 3.4],
    "B": [4.5, 5.6, 6.7],
    "C": [7.8, 8.9, 9.1]
```

Vishal Acharya

```
})

df8 = df8.astype(int)
print(df8)
```

```
   A  B  C
0  1  4  7
1  2  5  8
2  3  6  9
```

Cannot convert "abc" → int

❌ NaN cannot convert to int directly ✔ Use fillna() before astype() ✔ Use errors="ignore" to avoid crash

replace() in Pandas

Definition: replace() is used to replace values in a Series or DataFrame with new values.

It can replace:

Single values

Multiple values

Values using dictionary

Text patterns

Conditional replacements

Unlike map(), replace() works on Series AND DataFrame.

df.replace(to_replace=None, value=None, inplace=False, limit=None, regex=False, method=None) | Parameter | Meaning | | ---------- | ---------- | | `to_replace` | Value(s) or pattern(s) to replace | | `value` | New value(s) | | `inplace` | Modify original data | | `limit` | Maximum replacements per column | | `regex` | Use regular expression | | `method` | Forward/backward fill (`ffill` , `bfill`) |

Feature	replace()	map()
Works on	Series & DataFrame	Series only
Multiple values	✔	❌
Regex support	✔	❌
Conditional replace	✔	❌

1. Can replace numbers, strings, NaN
2. Supports regex replacement
3. Use limit to control replacements
4. inplace=True modifies original data
5. Safer than map() for bulk replacement

Vishal Acharya

Replace a Single Value

In [9]:
```
import pandas as pd

df1 = pd.DataFrame({
    "Marks": [80, 85, 0, 90],
    "Attendance": [75, 80, 0, 85],
    "Grade": ["A", "B", "F", "A"]
})

result = df1.replace(0, 60)
print(result)
```

```
   Marks  Attendance Grade
0     80          75     A
1     85          80     B
2     60          60     F
3     90          85     A
```

Replace Multiple Values (List)

In [293…
```
df2 = pd.DataFrame({
    "Score": [10, 20, 30, 10],
    "Level": ["Low", "Medium", "High", "Low"],
    "Status": ["P", "F", "P", "F"]
})

result = df2.replace([10, "Low"], [15, "Beginner"])
print(result)
```

```
   Score     Level Status
0     15  Beginner      P
1     20    Medium      F
2     30      High      P
3     15  Beginner      F
```

Replace Using Dictionary (Column-wise)

In [294…
```
df4 = pd.DataFrame({
    "Phone": ["+91-9876543210", "+91-9123456780"],
    "City": ["Ahmedabad123", "Surat456"],
    "Code": ["A1", "B2"]
})

result = df4.replace(r"\d+", "", regex=True)
print(result)
```

```
   Gender Result Dept
0    Male   Pass   IT
1  Female   Fail   HR
2    Male   Pass   IT
3  Female   Fail   HR
```

Replace Using Regular Expression

In [295…
```
df4 = pd.DataFrame({
    "Phone": ["+91-9876543210", "+91-9123456780"],
    "City": ["Ahmedabad123", "Surat456"],
    "Code": ["A1", "B2"]
})
```

Vishal Acharya

```
result = df4.replace(r"\d+", "", regex=True)
print(result)
```

```
  Phone       City Code
0    +-  Ahmedabad    A
1    +-      Surat    B
```

Replace with limit

Replaces only first occurrence per column

In [296…

```
df5 = pd.DataFrame({
    "Marks": [0, 0, 85, 0],
    "Attendance": [0, 75, 0, 80],
    "Grade": ["F", "F", "A", "B"]
})

result = df5.replace(0, 50, limit=1)
print(result)
```

```
   Marks  Attendance Grade
0     50          50     F
1     50          75     F
2     85          50     A
3     50          80     B
```

```
C:\Users\VISHAL\AppData\Local\Temp\ipykernel_25088\2671555998.py:7: FutureWarnin
g: The 'limit' keyword in DataFrame.replace is deprecated and will be removed in
a future version.
  result = df5.replace(0, 50, limit=1)
```

Filtering in Pandas means selecting rows or columns from a DataFrame based on conditions

Syntax: df[df["column"] condition value]

In [1]:

```
import pandas as pd

data = {
    "Name": ["A", "B", "C", "D"],
    "Age": [20, 25, 19, 30],
    "Marks": [80, 60, 90, 55]
}

df = pd.DataFrame(data)

result = df[df["Age"] > 21]
print(result)
```

```
  Name  Age  Marks
1    B   25     60
3    D   30     55
```

Filtering with Multiple Conditions

Vishal Acharya

Operator	Meaning
&	AND
`	OR
~	NOT

important Rule: Conditions must be inside parentheses.

In [2]:
```
df[(df["Age"] > 20) & (df["Marks"] > 70)]
```

Out[2]:

	Name	Age	Marks

In [3]:
```
df[(df["Age"] > 25) | (df["Marks"] > 85)]
```

Out[3]:

	Name	Age	Marks
2	C	19	90
3	D	30	55

Filtering Specific Columns

df.loc[condition , ["col1","col2"]]

In [4]:
```
df.loc[df["Marks"] > 70, ["Name", "Marks"]]
```

Out[4]:

	Name	Marks
0	A	80
2	C	90

In [5]:
```
df[(df["Age"] > 20) &
   (df["Marks"] > 60) &
   (df["Name"] != "B")]
```

Out[5]:

	Name	Age	Marks

Outlier Detection using IQR Method

What is an Outlier?

An outlier is a data point that is much larger or smaller than the rest of the dataset.

Student	Marks
A	45
B	50
C	55

Vishal Acharya

Student	Marks
D	60
E	62
F	65
G	200

Here 200 is an outlier.

Outliers can affect:

Mean

Statistical models

Machine learning predictions

So detecting them is important in data cleaning.

IQR Method Concept

IQR = Interquartile Range

$IQR = Q3 - Q1$ IQR=Q3−Q1

Where:

Q1 = 25th percentile

Q3 = 75th percentile

Outlier Boundaries

Lower Bound Q1−1.5×IQR

Upper Bound

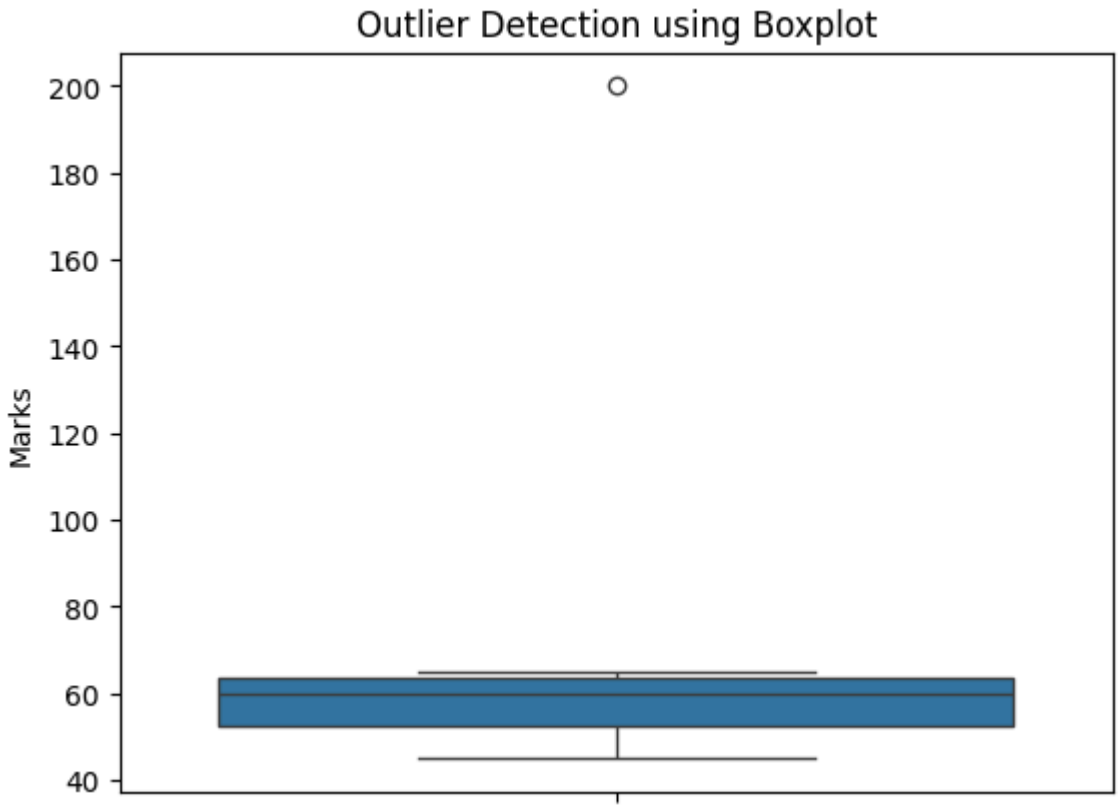
Q3+1.5×IQR

Any value outside this range is an outlier.

In [3]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Marks":[45,50,55,60,62,65,200]
}

df = pd.DataFrame(data)
sns.boxplot(y=df["Marks"])
plt.title("Outlier Detection using Boxplot")
plt.show()
```

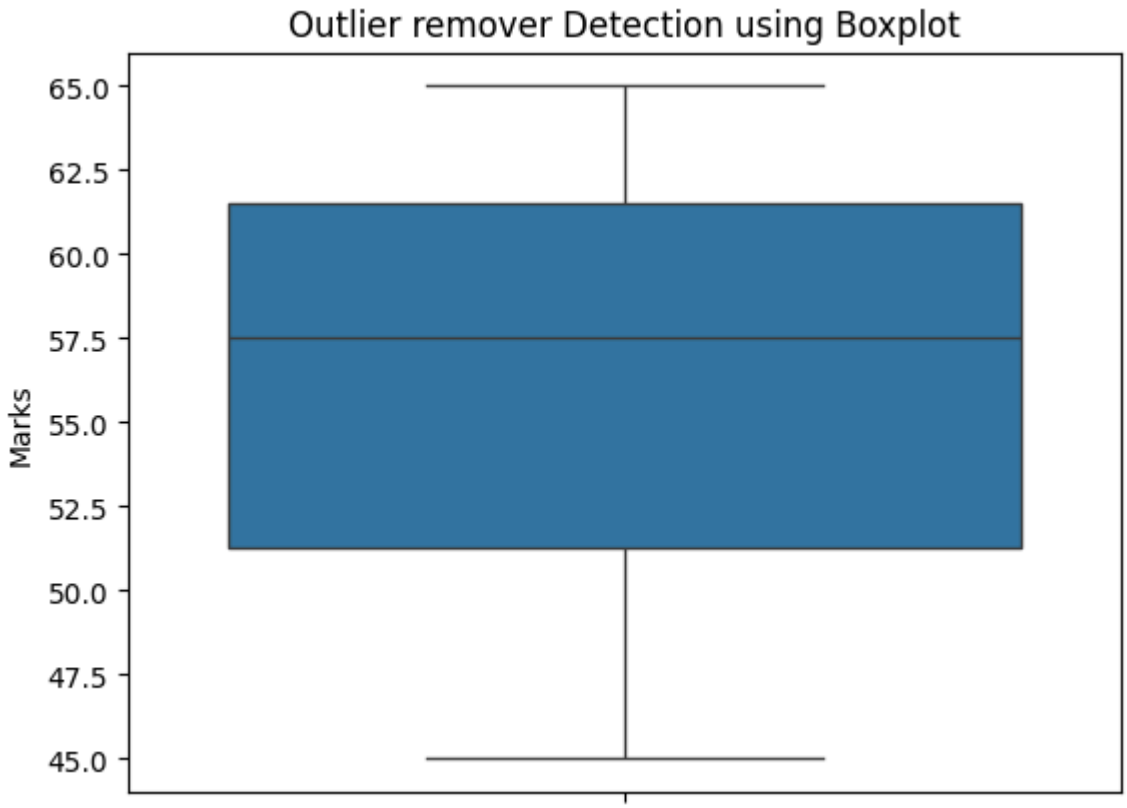
Vishal Acharya

```python
Q1 = df["Marks"].quantile(0.25)
Q3 = df["Marks"].quantile(0.75)

IQR = Q3 - Q1


lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["Marks"] < lower) | (df["Marks"] > upper)]

print("Outliers")
print(outliers)
print("remove outlier")
df=df[(df["Marks"] >= lower) & (df["Marks"] <= upper)]
sns.boxplot(y=df["Marks"])
plt.title("Outlier remover Detection using Boxplot")
plt.show()
```




```
Outliers
   Marks
6    200
remove outlier
```

Vishal Acharya



TASK

In [320…
```
data = pd.read_csv("PANDAS_DATA/batsman_runs_series.csv")
```

Find top 10 most run getter from the series.

In [321…
```
data.sort_values("batsman_run", ascending = False).head(10)
```

Out[321…

	batter	batsman_run
569	V Kohli	6634
462	S Dhawan	6244
130	DA Warner	5883
430	RG Sharma	5881
493	SK Raina	5536
27	AB de Villiers	5181
108	CH Gayle	4997
339	MS Dhoni	4978
452	RV Uthappa	4954
256	KD Karthik	4377

No of players having runs above 3000

In [322…
```
data[data["batsman_run"]>3000].shape[0]
```

Vishal Acharya

Out[322… 20

No of players having runs above mean value?

In [323…
```
data[data["batsman_run"] > data["batsman_run"].mean()].shape[0]
```

Out[323… 128

TASK2

1. Read `items.csv` making `item_name` as index.
2. Show no of nan values
3. Item price is given in $, so convert it to rupees without currency symbol.
4. Make data type of newly made series as float.
5. Fill nan with mean of the series

In [325…
```
#1
items = pd.read_csv("PANDAS_DATA/items.csv", index_col = ['item_name']).squeeze(
items
```

Out[325…
```
item_name
Chips and Fresh Tomato Salsa              $2.39
Izze                                      $3.39
Nantucket Nectar                          $3.39
Chips and Tomatillo-Green Chili Salsa     $2.39
Chicken Bowl                             $16.98
                                          ...
Steak Burrito                            $11.75
Steak Burrito                            $11.75
Chicken Salad Bowl                       $11.25
Chicken Salad Bowl                        $8.75
Chicken Salad Bowl                        $8.75
Name: item_price, Length: 4622, dtype: object
```

In [326…
```
#2
items.isna().sum()
```

Out[326… 50

In [327…
```
#2ii
def rupees(x):
    try:
        y = x[1:]
    except:
        # Sometimes Dollar sign is not there which throws an exception
        y = x
    return float(y)*82.49
items.apply(rupees)
```

Vishal Acharya

Out[327…

```
item_name
Chips and Fresh Tomato Salsa              197.1511
Izze                                      279.6411
Nantucket Nectar                          279.6411
Chips and Tomatillo-Green Chili Salsa     197.1511
Chicken Bowl                             1400.6802
                                            ...
Steak Burrito                             969.2575
Steak Burrito                             969.2575
Chicken Salad Bowl                        928.0125
Chicken Salad Bowl                        721.7875
Chicken Salad Bowl                        721.7875
Name: item_price, Length: 4622, dtype: float64
```

In [328…

```
#3
items = items.apply(rupees)
```

In [329…

```
#4
items.fillna(items.mean())
```

Out[329…

```
item_name
Chips and Fresh Tomato Salsa              197.1511
Izze                                      279.6411
Nantucket Nectar                          279.6411
Chips and Tomatillo-Green Chili Salsa     197.1511
Chicken Bowl                             1400.6802
                                            ...
Steak Burrito                             969.2575
Steak Burrito                             969.2575
Chicken Salad Bowl                        928.0125
Chicken Salad Bowl                        721.7875
Chicken Salad Bowl                        721.7875
Name: item_price, Length: 4622, dtype: float64
```

1. Find mean price
2. Find 30th and 6th percentile value
3. Plot Histogram on price with bin size 50
4. No of items price lies between [1000 to 2000]

In [331…

```
#1
items.mean()
```

Out[331…

```
615.6254681102363
```

In [333…

```
#2
print(items.quantile(q=0.3))
items.quantile(q=0.06)
```

```
367.0805
```

Out[333…

```
103.1125
```

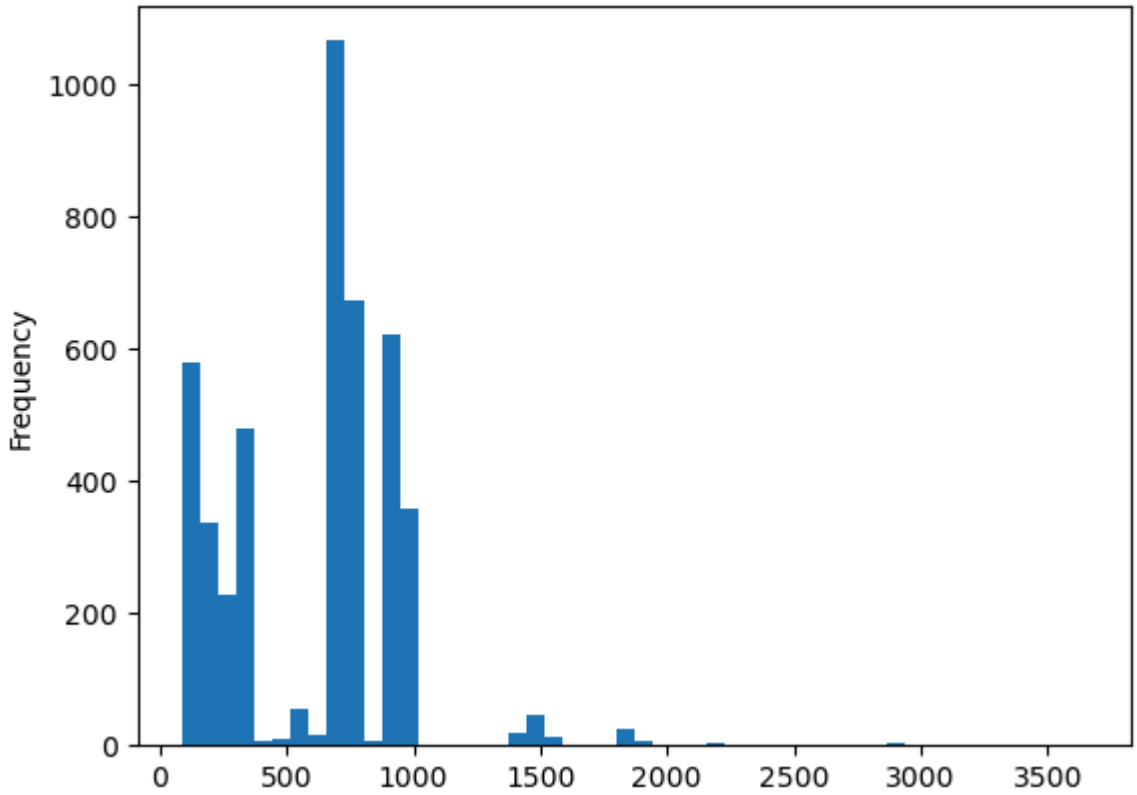
In [334…

```
#3
items.plot.hist(bins=50)
```

Out[334…

```
<Axes: ylabel='Frequency'>
```

Vishal Acharya



In [335…

```
items[(items>1000) & (items<2000)]
```

Out[335…

```
item_name
Chicken Bowl           1400.6802
Chicken Salad Bowl     1856.0250
Steak Burrito          1483.1702
Chicken Burrito        1443.5750
Chicken Bowl           1443.5750
                         ...
Chicken Bowl           1856.0250
Steak Bowl             1938.5150
Chicken Bowl           1443.5750
Chips and Guacamole    1468.3220
Chicken Salad Bowl     1443.5750
Name: item_price, Length: 116, dtype: float64
```

TASK 3

In [336…

```
data = {'birds': ['Cranes', 'Cranes', 'plovers', 'spoonbills', 'spoonbills', 'Cr
        'age': [3.5, 4, 1.5, np.nan, 6, 3, 5.5, np.nan, 8, 4, 3.5], 'visits': [2
        'priority': ['yes', 'yes', 'no', np.nan, 'no', 'no', 'no', 'yes', 'no',
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k']
```

1. Create a DataFrame birds from the above dictionary data which has the index labels.

2. Display basic information about the dataFrame.

3. Show Alternate rows of the dataframe.

In [337…

```
df1 = pd.DataFrame(data = data, index = labels)
```

Vishal Acharya

In [338…

```
#2
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 11 entries, a to k
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   birds     11 non-null     object
 1   age       9 non-null      float64
 2   visits    11 non-null     int64
 3   priority  10 non-null     object
dtypes: float64(1), int64(1), object(2)
memory usage: 440.0+ bytes
```

In [339…

```
df1.describe()
```

Out[339…

	age	visits
count	9.000000	11.000000
mean	4.333333	2.818182
std	1.903943	0.873863
min	1.500000	2.000000
25%	3.500000	2.000000
50%	4.000000	3.000000
75%	5.500000	3.500000
max	8.000000	4.000000

In [341…

```
#3
df1.iloc[::2]
```

Out[341…

	birds	age	visits	priority
a	Cranes	3.5	2	yes
c	plovers	1.5	3	no
e	spoonbills	6.0	3	no
g	plovers	5.5	2	no
i	spoonbills	8.0	3	no
k	Cranes	3.5	2	yes

i. Show only rows [1st, 3rd, 7th] from columns ['bird', 'age']

ii. Select rows where the number of visits is less than 4.

In [342…

```
#1
df1[['birds', 'age']].iloc[[0,2,6]]
```

Vishal Acharya

Out[342...

	birds	age
a	Cranes	3.5
c	plovers	1.5
g	plovers	5.5

In [343...

```
#2
df1[df1.visits<4]
```

Out[343...

	birds	age	visits	priority
a	Cranes	3.5	2	yes
c	plovers	1.5	3	no
e	spoonbills	6.0	3	no
g	plovers	5.5	2	no
h	Cranes	NaN	2	yes
i	spoonbills	8.0	3	no
j	spoonbills	4.0	2	no
k	Cranes	3.5	2	yes

i. Select all rows with nan values in age and visits column.

ii. Fill nan with respective series mode value.

In [344...

```
#1
df1[df1.age.isna() | df1.visits.isna()]
```

Out[344...

	birds	age	visits	priority
d	spoonbills	NaN	4	NaN
h	Cranes	NaN	2	yes

In [347...

```
df1.age=df1.age.fillna(df1.age.mode()[0])
df1.visits=df1.visits.fillna(df1.visits.mode()[0])
```

Find the total number of visits of the bird Cranes

ii. Find the number of each type of birds in dataframe.

iii. Print no of duplicate rows

iv. Drop Duplicates rows and make this changes permanent. Show dataframe after changes.

In [348...

```
#1
df1[df1.birds == "Cranes"].visits.sum()
```

Vishal Acharya

Out[348… 14

In [349…
```
#2
df1.birds.value_counts()
```

Out[349…
```
birds
Cranes        5
spoonbills    4
plovers       2
Name: count, dtype: int64
```

In [350…
```
#3
df1.duplicated().sum()
```

Out[350… 2

In [351…
```
#4
df1.drop_duplicates(inplace=True)
```

TASK 4

in IPL matches dataset some teams name has changed. You will have to consider them as same.

'Delhi Capitals' formerly as 'Delhi Daredevils' 'Punjab Kings' formerly as 'Kings XI Punjab' 'Rising Pune Supergiant' formerly as 'Rising Pune Supergiants' You need to make changes accordingly. Consider current name for each teams.

Be careful Gujrat Titans and Gujrat Lions are different teams.

In [3]:
```
data = pd.read_csv('PANDAS_DATA/IPL_Matches_2008_2022.csv')
changed_name = {'Delhi Daredevils':'Delhi Capitals',
                'Kings XI Punjab':'Punjab Kings',
                'Rising Pune Supergiants':'Rising Pune Supergiant'}
data.replace(changed_name.keys(), changed_name.values(),inplace=True )
```

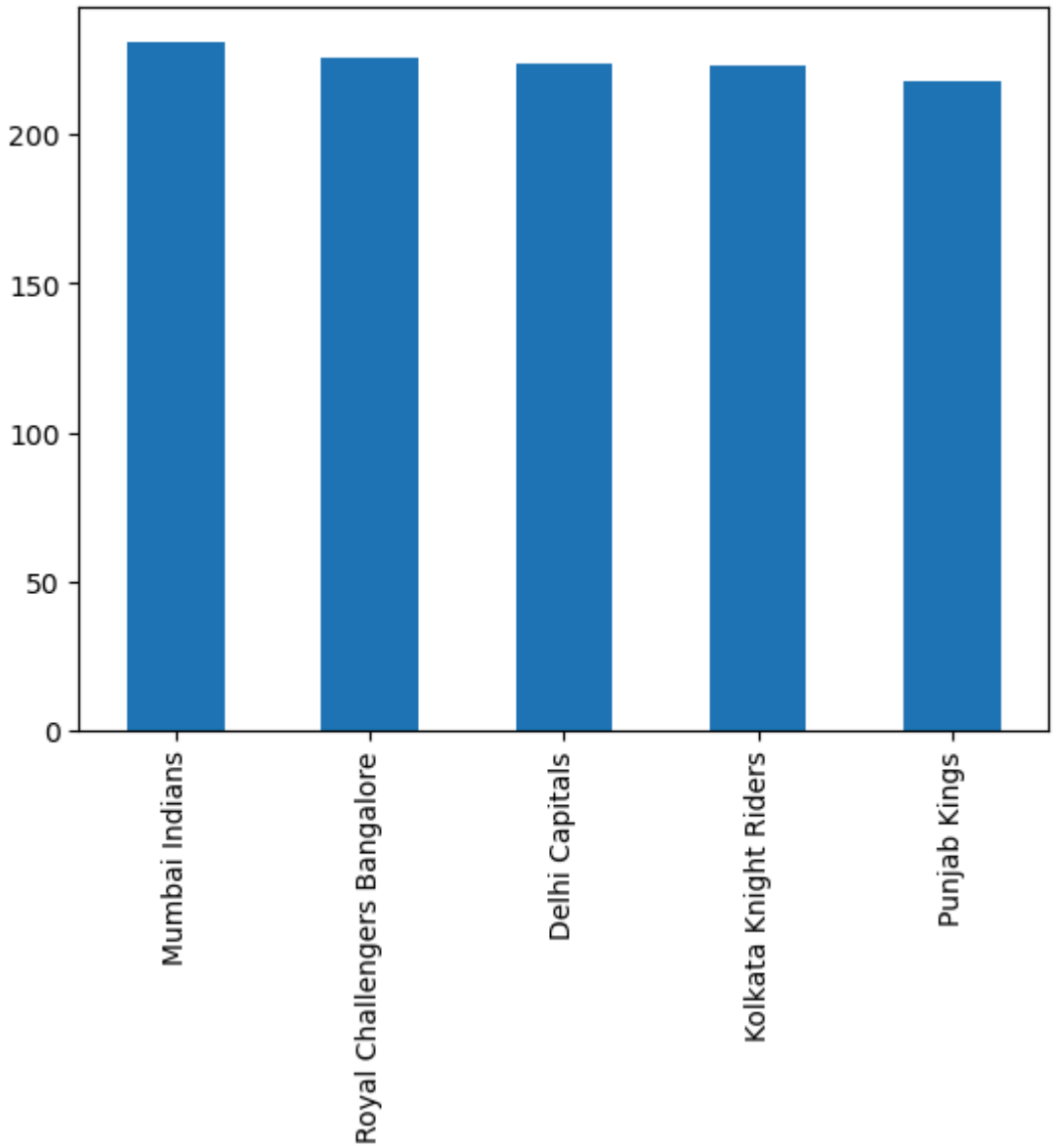
Write a code which can display the bar chart of top 5 teams who have played maximum number of matches in the IPL. Hint: Be careful the data is divided in 2 different cols(Team 1 and Team 2)

In [4]:
```
# Considering both team slots
(data['Team1'].value_counts()+data["Team2"].value_counts()).sort_values(ascendin
```

Out[4]: <Axes: >

Vishal Acharya

Vishal Acharya



Player who got Most no. of player of the match award playing against Mumbai Indians. Just for this question assume player of the match award is given to players from winning team. Although this is true in most of the cases.

In [5]:
```
m1 = (data.Team1 == "Mumbai Indians") | (data.Team2 == "Mumbai Indians")
m2 = data.WinningTeam != "Mumbai Indians"
data[m1 & m2].Player_of_Match.value_counts().head(1)
```

Out[5]:
```
Player_of_Match
SPD Smith    4
Name: count, dtype: int64
```

Team1 vs Team2 Dashbord Create a function which will take two string(name of two teams) as inMput. Show win Loss record between them and player getting most player of the match award in matches between these two teams.

In [6]:
```
def team1_vs_team2(t1, t2):
    m1 = (data.Team1 == t1) | (data.Team2 == t1)  # Filter for matches where onl
    m2 = (data.Team1 == t2) | (data.Team2 == t2)  # Filter for matches where onl
    df1 = data[m1 & m2]
    print(df1.WinningTeam.value_counts())
    print(df1.Player_of_Match.value_counts().head(1))
```

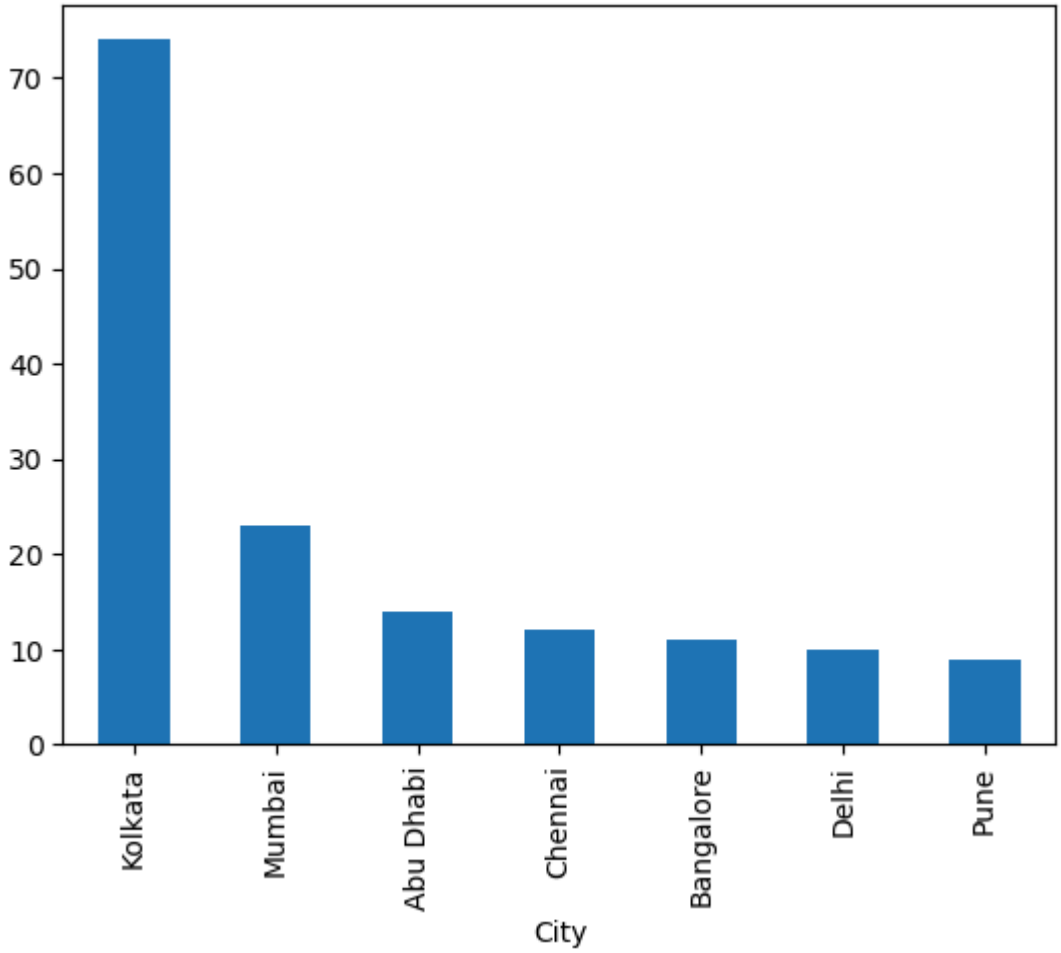
```
team1_vs_team2('Kolkata Knight Riders','Chennai Super Kings')
```

```
WinningTeam
Chennai Super Kings      17
Kolkata Knight Riders     9
Name: count, dtype: int64
Player_of_Match
RA Jadeja    3
Name: count, dtype: int64
```

Find out the top 7 cities where the matches of Kolkata Knight Riders are played frequently and plot the result as bar chart.

In [7]:
```
data[(data.Team1 == "Kolkata Knight Riders") | (data.Team2 == "Kolkata Knight Ri
```

Out[7]: <Axes: xlabel='City'>



Find out the average margin for the team Mumbai Indians for only the session 2011.

In [8]:
```
data[((data.Team1 == "Mumbai Indians") | (data.Team2 == "Mumbai Indians")) & (da
```

Out[8]: 19.25

find all the final winners

In [9]:
```
data[data['MatchNumber'] == 'Final'][['Season','WinningTeam']]
```

Vishal Acharya

Out[9]:

	Season	WinningTeam
0	2022	Gujarat Titans
74	2021	Chennai Super Kings
134	2020/21	Mumbai Indians
194	2019	Mumbai Indians
254	2018	Chennai Super Kings
314	2017	Mumbai Indians
373	2016	Sunrisers Hyderabad
433	2015	Mumbai Indians
492	2014	Kolkata Knight Riders
552	2013	Mumbai Indians
628	2012	Kolkata Knight Riders
702	2011	Chennai Super Kings
775	2009/10	Chennai Super Kings
835	2009	Deccan Chargers
892	2007/08	Rajasthan Royals

how many super over finishes have occured

In [11]:
```
data[data['SuperOver'] == 'Y'].shape[0]
```

Out[11]:
```
14
```

how many matches has csk won in kolkata

In [13]:
```
data[(data['City'] == 'Kolkata') & (data['WinningTeam'] == 'Chennai Super Kings'
```

Out[13]:
```
5
```

toss winner is match winner in percentage

In [15]:
```
(data[data['TossWinner'] == data['WinningTeam']].shape[0]/data.shape[0])*100
```

Out[15]:
```
51.473684210526315
```

find which player has won most potm -> in finals and qualifiers

In [16]:
```
data[~data['MatchNumber'].str.isdigit()]['Player_of_Match'].value_counts()
```

Vishal Acharya

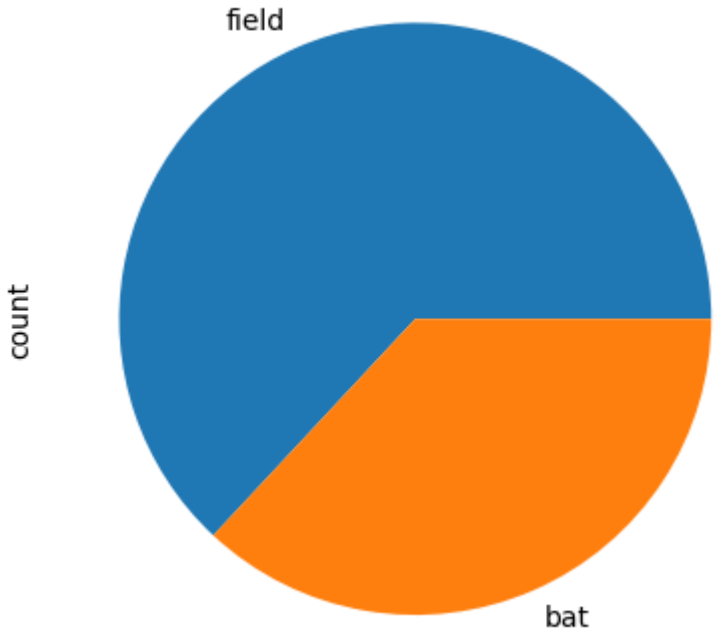
Out[16]:
```
Player_of_Match
KA Pollard           3
F du Plessis         3
SK Raina             3
A Kumble             2
MK Pandey            2
YK Pathan            2
M Vijay              2
JJ Bumrah            2
AB de Villiers       2
SR Watson            2
HH Pandya            1
Harbhajan Singh      1
A Nehra              1
V Sehwag             1
UT Yadav             1
MS Bisla             1
BJ Hodge             1
MEK Hussey           1
MS Dhoni             1
CH Gayle             1
MM Patel             1
DE Bollinger         1
AC Gilchrist         1
RG Sharma            1
DA Warner            1
MC Henriques         1
JC Buttler           1
RM Patidar           1
DA Miller            1
VR Iyer              1
SP Narine            1
RD Gaikwad           1
TA Boult             1
MP Stoinis           1
KS Williamson        1
RR Pant              1
SA Yadav             1
Rashid Khan          1
AD Russell           1
KH Pandya            1
KV Sharma            1
NM Coulter-Nile      1
Washington Sundar    1
BCJ Cutting          1
M Ntini              1
Name: count, dtype: int64
```

Toss decision plot

In [18]:
```
data['TossDecision'].value_counts().plot(kind='pie')
```

Out[18]:
```
<Axes: ylabel='count'>
```

Vishal Acharya



how many matches each team has played

In [20]:
```
(data['Team2'].value_counts() + data['Team1'].value_counts()).sort_values(ascend
```

Out[20]:
```
Mumbai Indians                 231
Royal Challengers Bangalore    226
Delhi Capitals                 224
Kolkata Knight Riders          223
Punjab Kings                   218
Chennai Super Kings            208
Rajasthan Royals               192
Sunrisers Hyderabad            152
Deccan Chargers                 75
Pune Warriors                   46
Gujarat Lions                   30
Rising Pune Supergiant          30
Gujarat Titans                  16
Lucknow Super Giants            15
Kochi Tuskers Kerala            14
Name: count, dtype: int64
```

Fifa Worldcup - 2022 dataset.

TASK 5

In [361…
```
df = pd.read_csv("PANDAS_DATA/Fifa Worldcup 2022.csv")
df.head()
```

Vishal Acharya

Out[361…

	Sl. No	Match No.	Team	Against	Group	Goal	Possession (%)	Inside Penalty Area	Outside Penalty Area	Assist
0	1	1	Qatar	Ecuador	A	0	40	0	0	(
1	2	1	Ecuador	Qatar	A	2	46	2	0	1
2	3	2	England	Iran	B	6	69	6	0	(
3	4	2	Iran	England	B	2	20	2	0	1
4	5	3	Senegal	Netherlands	A	0	39	0	0	(

5 rows × 38 columns

Use the football dataset. Find out the total percentages that each team made on target. Display the result as a python dictionary where the keys are the team list and the values are the percentage values. Round off the percentage values up to 2 decimal places.

- Help:

First, find out how many total teams are participated in this worldcup. For that, you can use unique() method on the column "Team" or "Against". Loop through the teams list that you have found in previous section, and then filter the dataset according to that. After filtering the dataset, find out total attempts sum and total on target sum. After getting these values, find out the percentage by total on target divided by total attempts and multiply by 100. And store to a python dictionary where the key will be the team name and the values will be the percentages. At the end,sort the dictionary by the values (not by the keys) and print the result.

In [362…

```
all_teams = df['Team'].unique()
on_target_percentage = {}
for team in all_teams:
    total_attempts = df[df['Team'] == team]["Total Attempts"].sum()
    total_on_target = df[df['Team'] == team]["On Target"].sum()
    on_target_percentage[team] = round(total_on_target / total_attempts * 100, 2
dict(sorted(on_target_percentage.items(), key = lambda x: x[1], reverse=True))
```

Vishal Acharya

Out[362…

```
{'Costa Rica': 54.55,
 'Cameroon': 51.85,
 'Ecuador': 48.15,
 'Argentina': 46.15,
 'Brazil': 45.56,
 'England': 45.0,
 'Portugal': 40.32,
 'Ghana': 40.0,
 'Netherlands': 39.02,
 'Korea Republic': 36.73,
 'Australia': 36.0,
 'Mexico': 34.88,
 'France': 34.65,
 'Germany': 34.33,
 'Croatia': 33.73,
 'Spain': 32.69,
 'Belgium': 32.35,
 'Serbia': 32.26,
 'Iran': 31.43,
 'Uruguay': 31.25,
 'United States': 31.11,
 'Saudi Arabia': 31.03,
 'Senegal': 30.77,
 'Denmark': 30.56,
 'Switzerland': 30.56,
 'Japan': 30.23,
 'Wales': 29.17,
 'Qatar': 28.57,
 'Morocco': 28.33,
 'Tunisia': 26.67,
 'Poland': 25.0,
 'Canada': 17.65}
```

find out how many times the teams are played in this Fifa Worldcup-2022. On top of this, find out the ranks of the teams.

Note: The DataFrame.rank() method takes an optiinal parameter named method. This parameter takes different values, but one of them is average which is by-default. So, when you do the rank, you will get some 2.5 like floating values. But if you change the value as first, then you will get in integers but the datatype will be float. So, try with method="first" parameter.

In [363…

```
df['Team'].value_counts().rank(method="first")
```

Vishal Acharya

Out[363…

```
Team
Morocco           29.0
Croatia           30.0
Argentina         31.0
France            32.0
England           25.0
Brazil            26.0
Netherlands       27.0
Portugal          28.0
Poland            17.0
Japan             18.0
Switzerland       19.0
Australia         20.0
Korea Republic    21.0
United States     22.0
Senegal           23.0
Spain             24.0
Cameroon           1.0
Uruguay            2.0
Belgium            3.0
Ghana              4.0
Canada             5.0
Qatar              6.0
Costa Rica         7.0
Germany            8.0
Ecuador            9.0
Mexico            10.0
Tunisia           11.0
Denmark           12.0
Saudi Arabia      13.0
Wales             14.0
Iran              15.0
Serbia            16.0
Name: count, dtype: float64
```

Find out these below topics:

1. The information about the Fifa worldcup dataset.
2. The description about the Fifa worldcup dataset
3. Check is there any missing values, if there is any missing values, fill that value with the average value for that particular column.
4. Drop all the duplicate rows permanently.
5. Drop the columns: "Sl No", "Match No.", "Red Cards" and "Pts" permanently.

In [364…

```
print(df.info())
print(df.describe())
print(df.isna().sum())
df.drop_duplicates(inplace=True)
df.drop(columns=["Sl. No", "Match No.", "Red Cards", "Pts"], inplace=True)
```

Vishal Acharya

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 128 entries, 0 to 127
Data columns (total 38 columns):
 #   Column                           Non-Null Count  Dtype
---  ------                           --------------  -----
 0   Sl. No                           128 non-null    int64
 1   Match No.                        128 non-null    int64
 2   Team                             128 non-null    object
 3   Against                          128 non-null    object
 4   Group                            128 non-null    object
 5   Goal                             128 non-null    int64
 6   Possession (%)                   128 non-null    int64
 7   Inside Penalty Area              128 non-null    int64
 8   Outside Penalty Area             128 non-null    int64
 9   Assists                          128 non-null    int64
 10  Total Attempts                   128 non-null    int64
 11  On Target                        128 non-null    int64
 12  Off Target                       128 non-null    int64
 13  Target in Penalty                128 non-null    int64
 14  Target from Outside              128 non-null    int64
 15  Left Channel                     128 non-null    int64
 16  Left Inside Channel              128 non-null    int64
 17  Central Channel                  128 non-null    int64
 18  Right Inside Channel             128 non-null    int64
 19  Right Channel                    128 non-null    int64
 20  Receptions MD                    128 non-null    int64
 21  Receptions D                     128 non-null    int64
 22  Attempted Line Breaks            128 non-null    int64
 23  Completed Line Breaks            128 non-null    int64
 24  Attempted Defensive Line Breaks  128 non-null    int64
 25  Completed Defensive Line Breaks  128 non-null    int64
 26  Yellow Cards                     128 non-null    int64
 27  Red Cards                        128 non-null    int64
 28  Fouls Against                    128 non-null    int64
 29  Offsides                         128 non-null    int64
 30  Passes                           128 non-null    int64
 31  Passes Completed                 128 non-null    int64
 32  Crosses                          128 non-null    int64
 33  Crosses Completed                128 non-null    int64
 34  Corners                          128 non-null    int64
 35  Free Kicks                       128 non-null    int64
 36  Penalties Scored                 128 non-null    int64
 37  Pts                              128 non-null    int64
dtypes: int64(35), object(3)
memory usage: 38.1+ KB
None
           Sl. No   Match No.        Goal  Possession (%)  \
count  128.000000  128.000000  128.000000      128.000000
mean    64.500000   32.500000    1.343750       43.476562
std     37.094474   18.545539    1.342448       11.442403
min      1.000000    1.000000    0.000000       14.000000
25%     32.750000   16.750000    0.000000       35.000000
50%     64.500000   32.500000    1.000000       45.000000
75%     96.250000   48.250000    2.000000       51.000000
max    128.000000   64.000000    7.000000       78.000000

       Inside Penalty Area  Outside Penalty Area     Assists  Total Attempts  \
count           128.000000            128.000000  128.000000      128.000000
mean              1.203125              0.117188    0.914062       11.257812
std               1.324270              0.322907    1.150333        5.406683
```

Vishal Acharya

```
min               0.000000              0.000000    0.000000        0.000000
25%               0.000000              0.000000    0.000000        7.750000
50%               1.000000              0.000000    1.000000       10.000000
75%               2.000000              0.000000    1.000000       14.000000
max               7.000000              1.000000    6.000000       32.000000

        On Target  Off Target  ...  Fouls Against    Offsides       Passes  \
count  128.000000  128.000000  ...     128.000000  128.000000   128.000000
mean     4.007812    4.835938  ...      12.320312    1.976562   502.859375
std      2.588576    2.610630  ...       4.691983    1.731891   160.160487
min      0.000000    0.000000  ...       0.000000    0.000000   225.000000
25%      2.000000    3.000000  ...       9.000000    1.000000   398.250000
50%      4.000000    4.000000  ...      12.000000    2.000000   495.500000
75%      6.000000    6.000000  ...      15.000000    3.000000   589.500000
max     13.000000   17.000000  ...      30.000000   10.000000  1070.000000

       Passes Completed     Crosses  Crosses Completed     Corners  \
count        128.000000  128.000000         128.000000  128.000000
mean         430.929688   19.523438           4.898438    4.468750
std          160.893724    8.596799           3.520111    2.774852
min          156.000000    4.000000           0.000000    0.000000
25%          323.250000   13.000000           3.000000    2.000000
50%          419.000000   19.000000           4.000000    4.000000
75%          519.750000   24.000000           6.000000    6.000000
max          994.000000   53.000000          25.000000   14.000000

       Free Kicks  Penalties Scored         Pts
count  128.000000        128.000000  128.000000
mean    14.250000          0.335938    1.429688
std      4.712189          0.941412    1.389973
min      5.000000          0.000000    0.000000
25%     11.000000          0.000000    0.000000
50%     14.000000          0.000000    1.000000
75%     17.000000          0.000000    3.000000
max     30.000000          5.000000    3.000000

[8 rows x 35 columns]
Sl. No                            0
Match No.                         0
Team                              0
Against                           0
Group                             0
Goal                              0
Possession (%)                    0
Inside Penalty Area               0
Outside Penalty Area              0
Assists                           0
Total Attempts                    0
On Target                         0
Off Target                        0
Target in Penalty                 0
Target from Outside               0
Left Channel                      0
Left Inside Channel               0
Central Channel                   0
Right Inside Channel              0
Right Channel                     0
Receptions MD                     0
Receptions D                      0
Attempted Line Breaks             0
```

Vishal Acharya

```
Completed Line Breaks              0
Attempted Defensive Line Breaks    0
Completed Defensive Line Breaks    0
Yellow Cards                       0
Red Cards                          0
Fouls Against                      0
Offsides                           0
Passes                             0
Passes Completed                   0
Crosses                            0
Crosses Completed                  0
Corners                            0
Free Kicks                         0
Penalties Scored                   0
Pts                                0
dtype: int64
```

Do these below operations:

1. Find out the rank based on the "Team" column and save the result by adding a new column named "Rank".
2. Change the datatype of this column to integer by using np.int16
3. Set the index of the DataFrame by using this "Rank" column permanently.
4. After that, sort the dataframe based on the "Rank" index.

In [365…

```
df["Rank"] = df["Team"].rank(method='first')
df["Rank"] = df["Rank"].astype(np.int16)
df.set_index("Rank", inplace=True)
df.sort_index()
```

Vishal Acharya

Out[365…

Rank	Team	Against	Group	Goal	Possession (%)	Inside Penalty Area	Outside Penalty Area	Assists	T Atten
1	Argentina	Saudi Arabia	C	1	51	1	0	0	
2	Argentina	Mexico	C	2	50	1	1	2	
3	Argentina	Poland	C	2	67	1	1	2	
4	Argentina	Australia	C	2	53	2	0	1	
5	Argentina	Netherlands	C	2	44	2	0	1	
...	...	...	...	...	...	...	...	...	
124	Uruguay	Portugal	H	0	35	0	0	0	
125	Uruguay	Ghana	H	2	39	2	0	1	
126	Wales	United States	B	1	39	1	0	0	
127	Wales	Iran	B	0	51	0	0	0	
128	Wales	England	B	0	33	0	0	0	

128 rows × 34 columns

TASK

Make a dataframe of each team in IPL with details like - Team Name, Matches Played, Win%, Home Win%, Away Win%. Show sorted dataframe on Win%

Replace old team name as new name before performing any tasks.

Delhi Daredevils ->Delhi Capitals Kings XI Punjab -> Punjab Kings Rising Pune Supergiants -> Rising Pune Supergiant

In [366…

```
matches = pd.read_csv("PANDAS_DATA/IPL_Matches_2008_2022.csv")
changed_name = {'Delhi Daredevils':'Delhi Capitals',
                'Kings XI Punjab':'Punjab Kings',
                'Rising Pune Supergiants':'Rising Pune Supergiant'}
matches.replace(changed_name.keys(), changed_name.values(),inplace=True )
```

In [367…

```
df = matches[~matches.WinningTeam.isna()]
new_df = pd.DataFrame()
teams = df.Team1.unique()
data = []
for team in teams:
    played = df[(df.Team1 == team) | (df.Team2 == team)].shape[0]
    wins = df[df.WinningTeam == team].shape[0]
    home_win = df[(df.WinningTeam == team) | (df.Team1 == team)].shape[0] / df[d
```

Vishal Acharya

```
    away_win = df[(df.WinningTeam == team) | (df.Team2 == team)].shape[0] / df[d
    data.append([team, played, wins/played*100, home_win, away_win])
new_df[["Team", "Matches Played", "Win Percentage", "Home Win Percentage", "Away
new_df.sort_values("Win Percentage", inplace=True, ascending=False)
new_df
```

Out[367…

	Team	Matches Played	Win Percentage	Home Win Percentage	Away Win Percentage
5	Gujarat Titans	16	75.0	214.285714	144.444444
6	Lucknow Super Giants	15	60.0	125.0	200.0
4	Chennai Super Kings	208	58.173077	150.45045	167.010309
9	Mumbai Indians	231	56.709957	158.928571	154.621849
7	Kolkata Knight Riders	223	51.121076	155.238095	147.457627
0	Rajasthan Royals	190	50.526316	152.808989	148.514851
10	Rising Pune Supergiant	30	50.0	157.142857	143.75
2	Sunrisers Hyderabad	152	49.342105	156.521739	143.373494
1	Royal Challengers Bangalore	223	48.878924	139.02439	161.0
3	Delhi Capitals	222	46.396396	144.247788	148.623853
8	Punjab Kings	218	46.330275	146.363636	146.296296
11	Gujarat Lions	30	43.333333	150.0	135.714286
14	Kochi Tuskers Kerala	14	42.857143	142.857143	142.857143
13	Deccan Chargers	75	38.666667	141.025641	136.111111
12	Pune Warriors	45	26.666667	126.086957	127.272727

Venues with most "no result" matches.

In [368…

```
matches[matches.WonBy == "NoResults"].Venue.value_counts().index[0]
```

Out[368…

```
'M Chinnaswamy Stadium'
```

Player with most appearance in final match. Team1Players and Team2Players have all players name. It is not a list of players name instead it is str. So handle it as string.

Hint: split and strip will help; Make a series of all players in final and do value counts

In [370…

```
import pandas as pd

def getPlayers(l):
    return [x.strip("'") for x in l.lstrip("'[").rstrip("]'").split(", ")]
```

Vishal Acharya

```
df_final = matches[matches["MatchNumber"] == "Final"]

players = []

for col in ["Team1Players", "Team2Players"]:
    for plist in df_final[col]:
        players.extend(getPlayers(plist))

playerSeries = pd.Series(players)

print(playerSeries.value_counts().head())
```

```
MS Dhoni     10
SK Raina      8
AT Rayudu     7
R Ashwin      7
DJ Bravo      7
Name: count, dtype: int64
```

IPL Point Table Make a function point_table which take season as parameter and show points table in non-ascendng order of points and in ascending order of team name.

For winning - 2 Ponits; For loosing - 0 Point For not result both team gets 1 points.

Make dataframe which will have TeamName MatchesPlayed MatchesWon NoResult Points make TeamName as index.

In [371…

```
# code here
def matches_played(df, team):
    return df[(df.Team1 == team) | (df.Team2 == team)].shape[0]
def matches_won(df, team):
    return df[df.WinningTeam == team].shape[0]
def matches_no_result(df, team):
    return df[((df.Team1 == team) | (df.Team2 == team)) & (df.WinningTeam.isnull

def point_table(season):
    df = matches[matches.Season == season]
    new_df = pd.DataFrame()
    new_df['Team Name'] = df.Team1.unique()
    new_df["Matches Played"] = new_df["Team Name"].apply(lambda x: matches_playe
    new_df["Matches Won"] = new_df["Team Name"].apply(lambda x: matches_won(df,
    new_df["No Result"] = new_df["Team Name"].apply(lambda x: matches_no_result(
    new_df["Points"] = new_df["Matches Won"]*2 + new_df["No Result"]
    new_df.sort_values("Points", ascending = False, inplace=True)
    new_df.set_index("Team Name", inplace=True)
    return new_df
point_table("2022")
```

Vishal Acharya

Out[371…

Team Name	Matches Played	Matches Won	No Result	Points
Gujarat Titans	16	12	0	24
Rajasthan Royals	17	10	0	20
Royal Challengers Bangalore	16	9	0	18
Lucknow Super Giants	15	9	0	18
Delhi Capitals	14	7	0	14
Punjab Kings	14	7	0	14
Sunrisers Hyderabad	14	6	0	12
Kolkata Knight Riders	14	6	0	12
Chennai Super Kings	14	4	0	8
Mumbai Indians	14	4	0	8

Vishal Acharya